

BookNest

Votre collection personnelle, comme une vraie bibliothèque

Bibliothèque personnelle unifiée — suivi, emprunts et lecture en un seul outil



PHP • MVC • PostgreSQL • Redis • Swagger

Résumé

BookNest est une application web de bibliothèque personnelle qui réunit dans un seul outil et une seule base de données trois usages habituellement dispersés : le catalogage des livres possédés, le suivi des prêts consentis à des proches, et la découverte de livres du domaine public lus directement dans l'application via l'API Gutendex (Project Gutenberg).

Le projet repose sur une architecture MVC maison en PHP 8.2 (sans framework), une base PostgreSQL 13, un cache Redis dont le client a été réimplémenté directement au-dessus d'un socket TCP (protocole RESP), une authentification JWT, et une conteneurisation Docker Compose complète (PHP-FPM, nginx, PostgreSQL, Redis). L'API est documentée via Swagger/OpenAPI généré depuis des annotations PHPDoc.

Ce dossier présente le cahier des charges, la conception Merise complète (MCD, MLD, MPD), l'architecture logicielle, la sécurité, les tests, la stratégie Git et le déploiement, avec des extraits de code réels commentés ligne à ligne.

Abstract

BookNest is a personal library web application unifying three usually separate concerns in a single tool and database: cataloguing owned books, tracking physical loans to friends and family, and discovering public-domain books read directly in-app via the Gutendex (Project Gutenberg) API. It is built on a hand-rolled PHP 8.2 MVC architecture, PostgreSQL 13, a Redis cache client re-implemented over a raw TCP socket (RESP protocol), JWT authentication, and a full Docker Compose containerisation (PHP-FPM, nginx, PostgreSQL, Redis), with the API documented via Swagger/OpenAPI.

Sommaire

Résumé	2
Abstract.....	2
Sommaire.....	3
Liste des figures et des tableaux	7
Figures	7
Glossaire	7
Partie I — Présentation générale	8
Chapitre 1 — Introduction générale.....	8
1.1 Présentation personnelle	8
1.2 Contexte du projet	8
1.3 Problématique.....	9
1.4 Objectifs	9
1.5 Méthodologie de travail	10
1.6 Gestion de projet — tableau Trello	11
Chapitre 2 — Étude préalable.....	12
2.1 Étude de l'existant	12
2.2 Critique de l'existant.....	12
2.3 Solution proposée : BookNest.....	13
2.4 Public cible et cas d'usage	13
Partie II — Cahier des charges	14
Chapitre 3 — Cahier des charges fonctionnel	14
3.1 Acteurs du système	14
3.2 Besoins fonctionnels	14
3.3 Besoins non-fonctionnels	15
3.4 Diagramme des cas d'utilisation.....	15
Chapitre 4 — Cahier des charges technique	16
4.1 Contraintes techniques	16
4.2 Choix technologiques justifiés.....	16
4.3 Environnements	18
Partie III — Conception Merise.....	20
Chapitre 5 — Dictionnaire de données.....	20
5.1 Recensement des entités	20
Chapitre 6 — MCD (Modèle Conceptuel de Données)	21
6.1 Règles de gestion.....	21
6.2 Diagramme MCD	21

6.3 Explication des entités et associations	21
Chapitre 7 — MLD (Modèle Logique de Données)	22
7.1 Règles de passage MCD → MLD	22
7.2 Schéma MLD.....	22
author, started_at, ended_at)	22
7.3 Description des relations.....	22
Chapitre 8 — MPD (Modèle Physique de Données)	23
8.1 Types PostgreSQL retenus	23
8.2 Schéma MPD	23
8.3 Index et contraintes.....	23
Chapitre 9 — Architecture MVC.....	24
9.2 Diagramme d'architecture	24
9.3 Flux d'une requête HTTP	24
9.3 Le routeur	25
25	
9.4 Modèles.....	25
26	
26	
Chapitre 10 — Structure des dossiers	27
10.1 Arborescence du projet.....	27
10.2 Rôle de chaque dossier	27
Chapitre 11 — Base de données PostgreSQL	28
11.1 Connexion et migrations	28
11.2 Transactions.....	28
Chapitre 12 — Cache Redis.....	29
12.1 Stratégie de cache	29
12.2 Un client RESP réimplémenté.....	29
Chapitre 13 — Sécurité et Authentification	30
13.1 JWT + session hybride	30
30	
13.2 Middleware de protection des routes.....	30
30	
13.3 Sécurité des données	30
Chapitre 14 — API et Documentation Swagger.....	31
14.1 Génération depuis les annotations	31
14.2 Endpoints principaux.....	32
Partie V — Front-end BookNest.....	33
Chapitre 15 — Direction artistique	33

15.1 Une identité assumée	33
15.2 Zoning.....	33
15.3 Wireframes.....	33
15.4 Palette de couleurs.....	35
15.5 Prototype.....	35
Chapitre 16 — Écrans principaux	38
16.1 Page d'accueil — étagères.....	38
16.2 Fiche livre	38
16.3 Lecteur immersif	39
16.4 Formulaire d'ajout.....	39
16.5 Page de connexion/Inscription.....	40
Partie VI — Qualité & Livraison	41
Chapitre 17 — Tests	41
17.1 Tests automatisés	41
41	
17.2 Cahier de recette	42
Chapitre 18 — Gestion Git.....	43
18.1 Organisation des branches	43
18.2 Convention de commits	43
Chapitre 19 — Déploiement	44
19.1 Docker Compose	44
19.2 Migrations versionnées	45
19.3 Hébergement en ligne et intégration continue	45
Partie VII — Bilan.....	46
Chapitre 20 — Difficultés rencontrées et solutions	46
20.1 Incompatibilité Docker Desktop / macOS	46
20.2 Bascule d'API externe pour le catalogue libre service	46
20.3 Cache navigateur masquant les mises à jour CSS	46
20.4 Perte de la base de données après renommage du projet	46
20.5 Doublons de livres importés depuis le catalogue	47
20.6 Lenteur perçue de la recherche et de la lecture	47
20.7 Cache Redis silencieusement inactif	47
20.8 Corruption du cache sur les grosses valeurs (texte de livre).....	47
Chapitre 21 — Perspectives d'évolution.....	48
Conclusion générale.....	49
Bibliographie	49

Liste des figures et des tableaux

Figures

- Figure 1 — MCD de BookNest
- Figure 2 — MLD de BookNest
- Figure 3 — MPD PostgreSQL et ERD
- Figure 4 — Architecture MVC
- Figure 5 — Zoning : connexion/inscription et bibliothèque
- Figure 6 — Zoning : catalogue libre service, fiche livre, lecteur
- Figure 7 — Wireframes : connexion, bibliothèque, catalogue, fiche livre
- Figure 8 — Wireframe du lecteur
- Figure 9 — Page de connexion
- Figure 10 — Page d'accueil (étagères)
- Figure 11 — Fiche livre (modal)
- Figure 12 — Catalogue libre service
- Figure 13 — Lecteur intégré
- Figure 14 — Résultats des tests unitaires
- Figure 15 — Interface Swagger UI

Glossaire

- MVC — Model, View, Controller. Séparation des responsabilités.
- PDO — PHP Data Objects. Couche d'accès aux bases de données.
- JWT — JSON Web Token. Authentification stateless signée.
- RESP — REdis Serialization Protocol. Protocole de communication avec Redis.
- TTL — Time To Live. Durée de vie d'une clé de cache.
- UUID — Universally Unique Identifier. Identifiant unique généré aléatoirement.
- ENUM — Type énuméré PostgreSQL, restreint à une liste de valeurs fixes.
- ORM — Object-Relational Mapping (non utilisé dans BookNest, accès direct via PDO).

Partie I — Présentation générale

Chapitre 1 — Introduction générale

1.1 Présentation personnelle

Je m'appelle Chaima Bekhaouda. J'ai obtenu un baccalauréat STI2D, spécialité SIN (Systèmes d'Information et Numérique), au lycée du Rempart. J'ai ensuite poursuivi vers une licence LLCER (Langues, Littératures et Civilisations Étrangères et Régionales), en anglais, arabe et chinois — un parcours volontairement éloigné de l'informatique à l'époque, mais qui m'a donné une aisance certaine à naviguer entre plusieurs langues et cultures, une compétence qui reste utile dans un secteur aussi internationalisé que le développement logiciel.

Après cette licence, j'ai travaillé pendant un an. C'est à ce moment que j'ai décidé de me réorienter vers l'informatique — un domaine qui m'a toujours attirée, et vers lequel je voulais depuis longtemps consacrer davantage de temps et d'énergie plutôt que de le laisser en simple curiosité personnelle.

Je suis aujourd'hui étudiante à La Plateforme, dans le cadre d'un Bachelor de trois ans, et alternante depuis deux ans chez Technip Energies, où j'apprends et je travaille en parallèle. Technip Energies est une entreprise française de technologies et d'ingénierie de premier plan dans les infrastructures énergétiques et la décarbonation (GNL, hydrogène, chimie durable, gestion du CO₂), employant plus de 18 000 collaborateurs dans 35 pays. Cette alternance m'a permis de confronter très tôt mes apprentissages théoriques à un contexte professionnel réel, au sein d'un grand groupe.

Je souhaite poursuivre mon parcours par un Master (MSc) à Epitech, où je suis déjà inscrite, toujours en alternance — mon entreprise actuelle a confirmé me garder pour les deux prochaines années dans ce cadre. À plus long terme, mon objectif est de poursuivre vers un doctorat à l'étranger, en lien avec Epitech.

1.2 Contexte du projet

Le marché du suivi de lecture est aujourd'hui dominé par des plateformes établies comme Goodreads, plus de 150 millions de membres, propriété d'Amazon, et par des alternatives plus récentes comme StoryGraph, qui a su séduire les lecteurs en quête d'un outil plus moderne et indépendant. Ce marché dynamique témoigne d'un besoin réel : les lecteurs veulent garder une trace de leur pratique, organiser leur collection et retrouver facilement ce qu'ils ont lu, aimé ou abandonné.

Pourtant, en observant l'usage réel de ces outils, un constat s'impose : tous se concentrent sur une seule facette de la vie d'un lecteur, le suivi de ce qu'on a lu, et délaissent le reste. La bibliothèque physique, elle, continue de vivre en dehors de toute application : les livres s'achètent, s'empilent, se rangent, et surtout se prêtent. Le prêt entre proches est une pratique universelle et une source tout aussi universelle de pertes : chacun a en tête un livre prêté « il y a un moment » dont on ne sait plus s'il a été rendu, ni même à qui il a été confié. Aucune des grandes plateformes ne couvre ce besoin pourtant élémentaire.

Enfin, un troisième usage reste systématiquement délégué à d'autres outils : la lecture elle-même. Les applications de suivi enregistrent qu'on lit, mais aucune ne permet de lire en son sein, il faut une liseuse, une application dédiée ou un site tiers, alors même que des dizaines

de milliers d'œuvres du domaine public sont librement disponibles en ligne. Le lecteur se retrouve ainsi à jongler entre trois univers étanches : son outil de suivi, sa bibliothèque physique dont personne ne tient le registre, et ses sources de lecture numérique. C'est dans cet interstice, trois usages réels, aucun outil unifié, que se situe BookNest.

1.3 Problématique

Comment réunir dans un seul outil trois usages qui vivent aujourd'hui dans des applications séparées, le catalogue personnel, le suivi des prêts physiques, et la lecture de livres du domaine public, sans sacrifier la rigueur technique attendue d'une application professionnelle ?

Cette question porte une double exigence. La première est fonctionnelle : l'unification n'a de valeur que si elle est réelle, c'est-à-dire si les trois usages partagent le même modèle de données, les mêmes statistiques et le même fil d'activité ; trois modules juxtaposés dans une même interface ne feraient que reproduire la fragmentation existante à plus petite échelle. La seconde est technique : le projet étant réalisé dans le cadre de la certification Concepteur Développeur d'Applications, il doit démontrer la maîtrise de l'ensemble de la chaîne de conception et de développement, architecture en couches, base de données relationnelle modélisée avec méthode, stratégie de cache, sécurité applicative, tests automatisés et documentation d'API. La problématique n'est donc pas seulement « quoi construire », mais « comment le construire dans les règles de l'art », chaque choix devant être justifiable

1.4 Objectifs

Les objectifs du projet découlent directement de la problématique et se déclinent en objectifs fonctionnels et techniques.

Sur le plan fonctionnel, BookNest doit d'abord offrir une étagère visuelle plutôt qu'un simple tableau de données : la bibliothèque numérique doit évoquer le plaisir d'une vraie bibliothèque, avec des couvertures et des étagères, et non l'austérité d'un back-office. Il doit permettre de gérer la collection de bout en bout : ajout manuel avec couverture, édition, organisation en étagères, et suivi du cycle de vie de chaque livre à travers quatre statuts (à lire, en cours, terminé, prêté). Il doit suivre les emprunts avec précision : savoir à tout moment qui détient quel livre, depuis quand, et s'il a été rendu — l'information exacte que la mémoire finit toujours par perdre. Il doit enfin donner accès à un catalogue de livres du domaine public directement lisibles dans l'application, faisant de BookNest non seulement un registre mais un véritable lieu de lecture.

Sur le plan technique, l'application doit exposer une API documentée via Swagger — condition d'une éventuelle ouverture future (application mobile, intégrations) et démonstration de la capacité à concevoir une interface machine propre. S'y ajoutent les objectifs transverses détaillés dans les besoins non-fonctionnels (chapitre 3.3) : performance perçue grâce au cache, sécurité par conception, et reproductibilité complète de l'environnement.

1.5 Méthodologie de travail

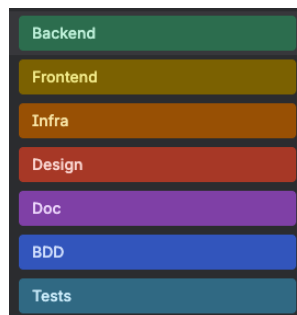
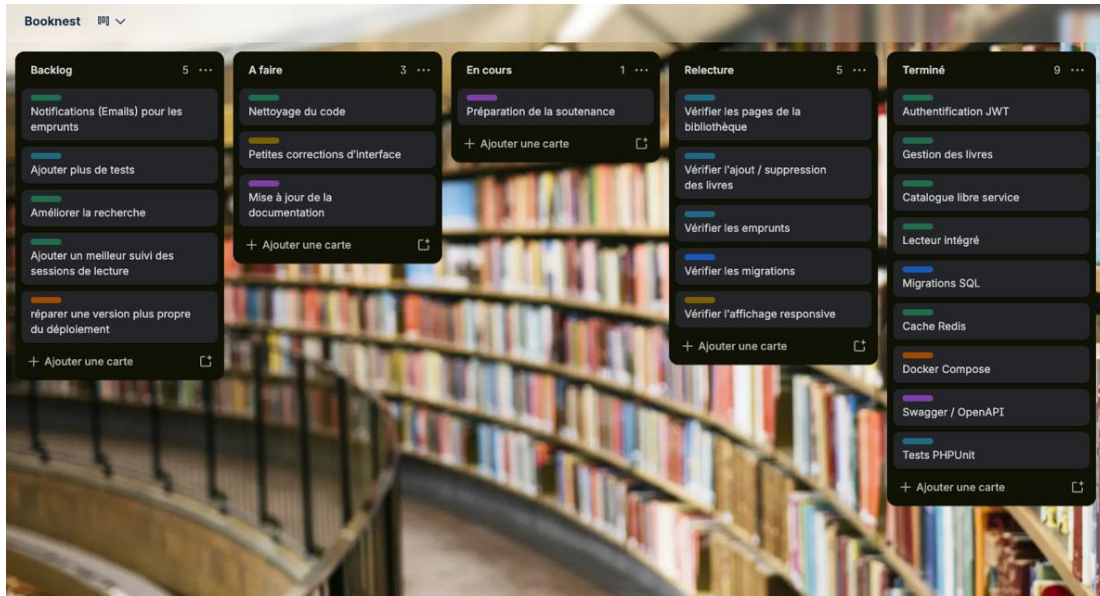
Le projet a été développé en solo, ce qui a orienté la méthodologie vers une adaptation pragmatique des principes agiles plutôt que vers un cadre formel (Scrum) dont les rituels — daily meetings, revues de sprint — n'ont pas de sens sans équipe. Le développement a procédé par itérations continues : chaque fonctionnalité (gestion des emprunts, catalogue libre-service, lecteur intégré, tableau de bord) a été construite de bout en bout, testée manuellement, puis affinée sur retour d'usage immédiat avant de passer à la suivante. Cette approche « une fonctionnalité à la fois, terminée avant la suivante » garantit qu'à tout moment l'application est dans un état démontrable — il n'y a jamais trois chantiers ouverts simultanément dont aucun ne fonctionne.

L'organisation du travail s'est appuyée sur un tableau Trello (chapitre 1.6) faisant office de backlog : les fonctionnalités y sont décrites, priorisées et suivies au fil de leur avancement, ce qui structure l'effort dans la durée et rend visible ce qui reste à faire. Le versioning Git reflète la même discipline : commits atomiques suivant la convention feat/fix/chore — chaque commit porte un changement unique et identifiable — sur des branches de fonctionnalité dédiées, fusionnées une fois la fonctionnalité stabilisée (chapitre 18).

Enfin, la conception a systématiquement précédé l'implémentation sur les points structurants : la base de données a été entièrement modélisée avec la méthode Merise (dictionnaire de données, MCD, MLD, MPD — Partie III) avant l'écriture de la moindre ligne de SQL, et les maquettes (zoning, wireframes — chapitre 15) ont fixé l'interface avant son intégration. Ce séquençement évite le piège classique du développement solo : coder d'abord et découvrir les incohérences de modèle ensuite, quand elles coûtent le plus cher à corriger.

1.6 Gestion de projet — tableau Trello

Le suivi des tâches s'est fait via un Trello à 5 colonnes (Backlog, À faire, En cours, Relecture, Terminé), avec des étiquettes par nature de tâche (Backend, Frontend, BDD, Design, Doc, Tests, Infra) :



Le tableau Trello structure le backlog du projet en colonnes suivant le flux de travail : les fonctionnalités à réaliser, priorisées par valeur (chapitre 1.5), celles en cours — limitées volontairement pour respecter le principe « une fonctionnalité terminée avant la suivante » — , et celles achevées, qui matérialisent l'avancement. Chaque carte correspond à un incrément fonctionnel et se retrouve dans l'historique Git sous forme d'une branche et d'une série de commits dédiés : le tableau et le dépôt racontent la même histoire à deux niveaux de détail. En solo, ce dispositif remplace la planification d'équipe : il externalise la liste mentale des tâches, rend visible le reste-à-faire, et évite la dispersion entre chantiers simultanés.

Chapitre 2 — Étude préalable

2.1 Étude de l'existant

Avant de concevoir BookNest, une étude des solutions existantes a été menée afin d'identifier ce qu'elles couvrent bien et ce qu'elles laissent de côté. Cinq applications représentatives du marché ont été analysées.

Goodreads, propriété d'Amazon, est la référence du domaine : son catalogue est immense et sa dimension sociale (avis, notes, listes partagées, défis de lecture) est très développée. Mais Goodreads est avant tout un réseau social du livre : il ne propose aucune gestion des prêts physiques, aucune lecture intégrée, et son interface, peu modernisée depuis des années, mélange la bibliothèque personnelle avec un flux communautaire que tous les lecteurs ne recherchent pas.

StoryGraph s'est imposé comme l'alternative moderne à Goodreads, avec un positionnement original : des recommandations fondées sur l'ambiance et le rythme d'un livre (« moods ») plutôt que sur la seule note en étoiles, et des statistiques de lecture très poussées. La limite reste la même : StoryGraph suit ce qu'on lit, pas ce qu'on possède ni ce qu'on prête, et n'offre aucun moyen de lire un texte dans l'application.

BookBuddy et Bookly abordent le problème sous un angle plus « gestionnaire » : catalogage fin de la collection physique (scan de code-barres, champs détaillés) pour le premier, suivi chronométré des sessions de lecture pour le second. Le suivi des prêts y existe, mais reste une fonctionnalité secondaire. Surtout, la lecture elle-même est explicitement hors périmètre — Bookly précise dans sa propre description que le contenu du livre n'est pas géré dans l'application.

Fable, enfin, propose un lecteur intégré de qualité, mais réservé à un contenu sous licence payante via son partenariat Everand : il s'agit d'un modèle d'abonnement à un catalogue fermé, pas d'un accès au domaine public, et la gestion d'une collection physique personnelle n'y a pas sa place.

x2.2 Critique de l'existant

Cette étude fait apparaître trois constats structurants. Premièrement, aucune plateforme ne combine dans un même outil les trois dimensions de la vie d'un lecteur : le catalogue de ce qu'il possède, le suivi de ce qu'il a prêté, et la lecture effective. Chaque application excelle sur un axe et ignore les deux autres. Deuxièmement, le suivi des prêts — pourtant une source classique de livres perdus entre amis — est soit totalement absent des grandes plateformes, soit relégué au rang de fonctionnalité annexe dans les applications de catalogage. Troisièmement, il n'existe aucune passerelle entre ce qu'on possède, ce qu'on a prêté et ce qu'on lit gratuitement en ligne : chaque source de lecture vit dans un outil différent, avec des statistiques dispersées et aucune vue d'ensemble. Le lecteur qui veut tout suivre doit jongler entre deux ou trois applications qui s'ignorent mutuellement.

2.3 Solution proposée : BookNest

BookNest ne cherche pas à concurrencer Goodreads sur l'échelle de son catalogue ni StoryGraph sur les statistiques sociales : son positionnement est volontairement différent et complémentaire. Sa proposition de valeur tient en un mot, l'unification. Qu'un livre soit ajouté manuellement à sa collection, marqué comme prêté à un proche, ou importé depuis le catalogue libre-service du domaine public, il atterrit dans la même table de la même base de données, alimente les mêmes statistiques et le même fil d'activité. Il en résulte une trace complète et cohérente de la vie de lecteur de l'utilisateur : ce qu'il possède, où se trouvent ses livres, ce qu'il a lu et ce qu'il est en train de lire — y compris directement dans l'application, grâce au lecteur intégré adossé au domaine public via l'API Gutendex. Ce choix d'un périmètre resserré mais entièrement couvert est aussi ce qui rend le projet réalisable et démontrable dans le cadre de la certification : chaque fonctionnalité annoncée est effectivement implémentée, testée et déployée.

2.4 Public cible et cas d'usage

BookNest s'adresse aux lecteurs particuliers gérant leur bibliothèque personnelle, sans dimension communautaire : pas d'amis, pas d'avis publics, pas de flux social — l'application est pensée comme un outil privé, ce qui la distingue frontalement des plateformes existantes et simplifie d'ailleurs son modèle de données (un seul rôle utilisateur, chapitre 3.1). Trois cas d'usage concrets illustrent le quotidien visé. Le catalogage : après un déménagement ou face à une bibliothèque qui déborde, recenser sa collection, l'organiser en étagères et retrouver un titre en quelques secondes. Le prêt : prêter un roman à un ami en notant son nom et la date, puis, six mois plus tard, savoir immédiatement qui détient quel livre au lieu de compter sur sa mémoire. La découverte : chercher un classique du domaine public — Jules Verne, Jane Austen, Victor Hugo — et le lire directement dans l'application, sans installer de liseuse dédiée ni créer de compte sur un service tiers, le livre rejoignant automatiquement la bibliothèque et les statistiques de lecture.

Partie II — Cahier des charges

Chapitre 3 — Cahier des charges fonctionnel

3.1 Acteurs du système

BookNest est pensé pour un utilisateur unique par compte : le lecteur, propriétaire de sa collection. Ce choix, qui peut sembler minimaliste, est en réalité structurant pour toute la conception. Il n'existe pas de rôle visiteur public : aucune donnée n'est consultable sans authentification, chaque page et chaque endpoint de l'API étant protégés par le middleware d'authentification. Il n'existe pas non plus de rôle administrateur distinct : l'application ne gère ni modération, ni contenu partagé entre comptes, ce qui rendrait ce rôle sans objet dans le périmètre actuel. Enfin, un choix de modélisation important en découle : l'emprunteur d'un livre n'est pas un acteur du système. C'est une personne physique du monde réel (un ami, un membre de la famille) représentée par un simple champ texte libre dans l'entité EMPRUNT — elle ne se connecte pas, n'a pas de compte, et n'interagit jamais directement avec l'application. Cette décision évite de forcer les proches de l'utilisateur à créer un compte pour une action qui ne les concerne pas, et simplifie considérablement le modèle de données (chapitre 6). La conséquence est un système à un seul rôle, où toute la logique d'autorisation se résume à une règle unique et vérifiable : un utilisateur n'accède qu'à ses propres données.

3.2 Besoins fonctionnels

Les besoins fonctionnels se répartissent en quatre familles cohérentes avec les cas d'usage identifiés au chapitre 2.4.

Gestion du compte — L'utilisateur doit pouvoir s'inscrire avec un email, un mot de passe et un nom d'affichage, se connecter, et se déconnecter. L'email est unique dans le système : il sert d'identifiant de connexion. Toute tentative d'accès à une page protégée sans session valide doit rediriger vers la page de connexion.

Gestion de la collection — L'utilisateur doit pouvoir ajouter un livre manuellement (titre et auteur obligatoires, couverture optionnelle uploadée depuis son appareil), modifier son statut de lecture parmi quatre valeurs (à lire, en cours, terminé, prêté), et le supprimer — la suppression retirant également la couverture stockée sur le serveur. La collection s'organise en étagères paginées, avec la possibilité d'ajouter des étagères supplémentaires ; la recherche et le filtrage par genre et par statut permettent de retrouver un livre rapidement dans une collection volumineuse.

Gestion des emprunts — L'utilisateur doit pouvoir marquer un livre comme prêté en indiquant le nom de l'emprunteur, ce qui crée un emprunt daté et bascule automatiquement le statut du livre à « prêté » — les deux opérations étant indissociables (transaction, chapitre 11.2). Il doit pouvoir marquer le retour, ce qui clôt l'emprunt (date de retour renseignée) et libère le livre. La liste des emprunts en cours doit être consultable d'un coup d'œil, car c'est précisément le « qui a quoi » qui motive la fonctionnalité.

Découverte et lecture — L'utilisateur doit pouvoir rechercher un livre du domaine public via le catalogue libre-service (API Gutendex), l'ajouter à sa bibliothèque, et le lire directement dans l'application via un lecteur paginé. L'ouverture d'un livre démarre une session de lecture horodatée, alimentant le fil d'activité. Enfin, un tableau de bord présente les statistiques de la collection (répartition par statut, activité récente) pour donner cette vue d'ensemble absente des solutions existantes.

3.3 Besoins non-fonctionnels

Performance — Les pages les plus consultées (accueil, tableau de bord) doivent rester fluides même lorsque leur construction est coûteuse : les statistiques et listes fréquemment lues sont mises en cache (Redis, TTL de 5 minutes), et les appels à l'API externe Gutendex — le poste de latence le plus élevé de l'application — sont systématiquement mis en cache (10 minutes pour les recherches, 24 heures pour les métadonnées). Le changement de page dans le lecteur ne doit déclencher aucun appel réseau.

Sécurité — Aucune donnée n'est accessible sans authentification (JWT signé HS256). Les mots de passe sont hachés via `password_hash` (bcrypt), jamais stockés ni journalisés en clair. Toutes les requêtes SQL passent par des requêtes préparées PDO, et chaque requête intègre la vérification d'appartenance (`user_id`) pour interdire tout accès horizontal aux données d'un autre compte. Les secrets de configuration sont externalisés et non versionnés.

Ergonomie — L'interface doit être responsive (consultable sur mobile, où l'on vérifie typiquement « qui a mon livre » ou l'on lit dans les transports) et porter une identité visuelle propre — l'univers chaleureux d'une bibliothèque — plutôt que l'apparence générique d'un CRUD d'exercice. Les actions courantes (changer un statut, prêter, rendre) doivent tenir en un ou deux clics depuis l'étagère.

Maintenabilité — L'architecture en couches strictes (Controller / Repository / Service) garantit que chaque type de modification a un emplacement unique et prévisible. Les composants à logique propre sont couverts par des tests unitaires automatisés, exécutés en intégration continue avant tout déploiement.

Portabilité — L'environnement complet (serveur web, PHP, base, cache) est conteneurisé via Docker Compose : le projet s'installe sur toute machine en une commande, sans dépendre des versions logicielles installées localement — condition de la reproductibilité entre la machine de développement, un éventuel autre poste, et la production.

3.4 Diagramme des cas d'utilisation

Le diagramme s'articule autour d'un acteur unique, l'utilisateur authentifié, et de trois grandes familles de cas d'utilisation qui recoupent directement les besoins fonctionnels. La gestion de la collection regroupe l'ajout manuel d'un livre, le changement de statut, l'organisation en étagères et la suppression. La gestion des emprunts comprend le prêt (qui inclut systématiquement le changement de statut du livre — relation d'inclusion) et le retour. La découverte/lecture couvre la recherche dans le catalogue libre-service, l'import d'un livre externe et la lecture paginée, cette dernière déclenchant l'enregistrement d'une session de lecture. L'inscription et la connexion précèdent l'ensemble : tous les cas d'utilisation métier ont pour précondition une session authentifiée, matérialisée sur le diagramme par la dépendance de chaque famille envers le cas « S'authentifier ».

Chapitre 4 — Cahier des charges technique

4.1 Contraintes techniques

Les contraintes suivantes définissent le socle imposé de l'environnement d'exécution, chaque élément étant figé en version pour garantir la reproductibilité.

PHP 8.2-FPM (image Docker officielle) constitue le moteur d'exécution. Les extensions requises sont installées à la construction de l'image : `pdo_pgsql` (accès PostgreSQL via PDO), `mbstring` (manipulation fiable des chaînes multi-octets — les textes de Gutendex contiennent des caractères accentués et typographiques), `zip` et `gd` (manipulation d'archives et d'images pour les couvertures uploadées), `opcache` (compilation en cache des scripts PHP, gain de performance significatif en production), et `redis` via `PECL` — cette dernière étant présente dans l'image bien que le client applicatif soit réimplémenté (chapitre 12), ce qui a permis de comparer les deux approches pendant le développement.

PostgreSQL 13 est la base de données primaire et l'unique source de vérité : toute donnée métier y réside. La version 13 apporte `gen_random_uuid()` via `pgcrypto` et l'extension `pg_trgm` (installée en prévision de la recherche floue, chapitre 21).

Redis 7 est strictement cantonné au rôle de cache : aucune donnée n'y existe sans exister d'abord en PostgreSQL, et son indisponibilité doit dégrader les performances, jamais les fonctionnalités.

nginx (image stable-alpine, choisie pour sa légèreté) est le seul point d'entrée HTTP : il sert les fichiers statiques et relaie les requêtes PHP à FPM via FastCGI, en n'exposant que le dossier `public/`.

Composer gère les dépendances, volontairement réduites à trois bibliothèques : `firebase/php-jwt` (signature et validation des tokens), `zircote/swagger-php` (génération de la documentation OpenAPI) et `phpunit/phpunit` (tests). Ce nombre restreint est une contrainte assumée : chaque dépendance évitée est une surface d'attaque, une source de mise à jour et une boîte noire en moins — en cohérence avec la démarche « comprendre plutôt que dépendre » qui traverse le projet.

4.2 Choix technologiques justifiés

Chaque brique du projet a été retenue pour une raison précise. Cette section détaille, technologie par technologie, ce qu'elle est, le rôle qu'elle joue dans BookNest et pourquoi elle a été préférée à ses alternatives.

PHP 8.2 (langage back-end) — PHP est un langage conçu dès l'origine pour le web côté serveur : chaque requête HTTP démarre un traitement isolé qui se termine avec la réponse, un modèle simple à raisonner et naturellement étanche — une requête qui échoue n'affecte jamais les autres. Son mode d'exécution PHP-FPM (FastCGI Process Manager) maintient un pool de processus réutilisables, évitant le coût de démarrage à chaque requête. La version 8.2 apporte un typage strict moderne — propriétés typées, types union, propriétés `readonly`, `enums` — qui permet d'écrire un PHP rigoureux, loin de l'image permissive du langage : la majorité des erreurs de type sont détectées à l'exécution immédiate plutôt que de se propager silencieusement. Le choix de ne pas utiliser de framework (Laravel, Symfony) est le choix structurant du projet, détaillé au chapitre 9 : construire soi-même le routage, le cycle

requête/réponse et les middlewares permet de comprendre chaque mécanisme que les frameworks masquent. La contrepartie est assumée — pas de segments de route dynamiques, pas d'ORM — et documentée là où elle a orienté la conception (chapitre 9.3).

SQL et PostgreSQL 13 (base de données) — Les données de BookNest sont naturellement relationnelles : un utilisateur possède des livres, un livre fait l'objet d'emprunts et de sessions de lecture. Ces relations s'expriment par des clés étrangères et se protègent par des contraintes d'intégrité — le terrain exact des bases relationnelles, là où une base NoSQL orientée documents obligerait à réimplémenter ces garanties dans le code applicatif. PostgreSQL a été préféré à MySQL/MariaDB, pourtant plus courant dans l'écosystème PHP, pour son typage strict et sa richesse : type UUID natif avec génération côté base (`gen_random_uuid()`), types ENUM personnalisés qui font porter les règles métier par la donnée elle-même, TIMESTAMPTZ pour des horodatages fiables indépendamment des fuseaux, clause RETURNING qui économise un aller-retour après insertion (chapitre 9.4), et un respect rigoureux du standard SQL. Ses propriétés ACID, exploitées via les transactions PDO, garantissent qu'aucun état partiel ne subsiste en cas d'erreur (chapitre 11.2).

Redis 7 (cache) — Redis est une base clé-valeur résidant entièrement en mémoire vive : une lecture s'y compte en microsecondes, contre plusieurs millisecondes pour une requête SQL et plusieurs centaines pour un appel à une API externe. Dans BookNest, il est strictement cantonné au rôle de cache (stratégie cache-aside avec TTL différenciés, chapitre 12.1) : PostgreSQL reste l'unique source de vérité, et l'indisponibilité de Redis dégrade les performances, jamais les fonctionnalités. Plutôt que d'utiliser l'extension PECL redis comme une boîte noire, le client a été réimplémenté au-dessus d'un socket TCP brut en suivant le protocole RESP : un choix pédagogique qui a d'ailleurs confronté le projet à une vraie difficulté de programmation réseau — la lecture partielle des gros payloads (chapitre 20.8) — qu'une bibliothèque toute faite aurait masquée.

HTML5, CSS3 et JavaScript vanilla (front-end) — Les pages sont rendues côté serveur par les vues PHP, en HTML sémantique. Le CSS est écrit à la main, sans framework type Bootstrap ou Tailwind, afin de construire l'identité visuelle propre de BookNest — l'univers chaleureux d'une bibliothèque — plutôt qu'une interface générique reconnaissable entre mille. Le JavaScript est en vanilla, sans React ni Vue ni chaîne de build : les besoins d'interactivité sont ciblés (modales, filtres, appels fetch vers l'API pour le lecteur paginé) et un framework front aurait imposé Node, un bundler et des dizaines de dépendances npm pour un bénéfice nul à cette échelle. Le vanilla démontre au contraire la maîtrise directe du DOM et des API du navigateur — la couche que les frameworks abstraient.

nginx (serveur web) — nginx est un serveur web léger et très performant, standard de fait devant PHP-FPM. Il joue deux rôles : servir directement les fichiers statiques (CSS, images, JavaScript, couvertures uploadées), ce qu'il fait bien plus efficacement que PHP, et relayer les requêtes dynamiques au conteneur FPM via le protocole FastCGI (port 9000). Seul le dossier public/ est exposé : le code applicatif, les migrations et la configuration sont physiquement inaccessibles par URL, quelle que soit la requête. C'est aussi ce découpage qui explique l'architecture à deux conteneurs : PHP-FPM ne parle pas HTTP — sans nginx devant lui, aucune page ne peut être servie.

Composer (gestion des dépendances) — Composer est le gestionnaire de dépendances standard de PHP : il installe, versionne (`composer.lock`) et charge automatiquement (autoloading PSR-4) les bibliothèques tierces. Le fichier lock garantit que chaque environnement — machine de développement, CI, production — installe exactement les mêmes versions, au patch près.

JWT (authentication) — Le JSON Web Token est un jeton signé et autoporté : il contient l'identité de l'utilisateur et sa date d'expiration, vérifiables par la seule signature HS256, sans

consulter la base. Modifier son contenu invalide mathématiquement la signature. BookNest le combine à une session PHP reconstruite à chaque requête — le choix hybride et ses raisons sont détaillés au chapitre 13.1.

Swagger / OpenAPI (documentation d'API) — La documentation est générée depuis des annotations PHPDoc placées directement dans les contrôleurs : elle vit au plus près du code qu'elle décrit et reste synchronisée avec lui. OpenAPI étant le standard industriel, le fichier `openapi.json` produit est exploitable par des machines (import Postman, génération de clients), et Swagger UI le rend interactif : chaque endpoint s'essaie depuis le navigateur, faisant de la documentation un outil de test manuel utilisé pendant tout le développement.

API Gutendex (catalogue libre-service) — Gutendex expose le catalogue du Project Gutenberg — plus de 70 000 œuvres du domaine public — via une API REST gratuite, sans clé ni quota, donnant accès au texte intégral brut. Ce dernier point est décisif : il a permis de développer un lecteur paginé entièrement contrôlé par l'application, plutôt que de dépendre d'un widget tiers. Ce choix est le résultat d'une bascule en cours de projet depuis l'API Google Books, documentée au chapitre 20.2.

Alternatives écartées — Trois grandes familles d'alternatives ont été consciemment écartées. Node.js ou Python (Django/Flask) pour le back-end : PHP correspond au cœur du référentiel de formation, son modèle « une requête = un traitement isolé » est plus simple à raisonner qu'une boucle d'événements, et son hébergement est universellement supporté. MySQL/MariaDB : écarté au profit du typage strict de PostgreSQL. Un framework front (React, Vue) : écarté pour garder l'architecture simple et le nombre de technologies réellement maîtrisées volontairement resserré — un critère de qualité pour un projet individuel de certification, où mieux vaut peu de briques comprises en profondeur que beaucoup de briques survolées.

4.3 Environnements

Pourquoi Docker — Docker exécute chaque composant de l'application dans un conteneur : un processus isolé embarquant exactement le logiciel et la version dont il a besoin, tout en partageant le noyau du système hôte — bien plus léger qu'une machine virtuelle par service. BookNest en orchestre quatre : nginx, PHP-FPM 8.2, PostgreSQL 13 et Redis 7. Les bénéfices sont concrets et se sont vérifiés tout au long du projet : environnement reproductible à l'identique sur n'importe quelle machine (fini le « ça marche chez moi »), versions figées et documentées dans le code lui-même, isolation totale vis-à-vis de la machine hôte (le PostgreSQL du projet ne peut entrer en conflit avec aucune autre installation), et parité entre développement et production, le déploiement sur Render reposant lui aussi sur une image Docker construite depuis le même socle.

Pourquoi Docker Compose — Docker Compose décrit ces quatre conteneurs dans un unique fichier déclaratif, `docker-compose.yml` : services, ports exposés, volumes de persistance (les données PostgreSQL survivent à la destruction et à la recreation du conteneur) et réseau interne y sont définis une fois pour toutes. Sur ce réseau interne, le nom d'un service tient lieu d'adresse (`REDIS_HOST=redis`, `DB_HOST=db`) — un mécanisme dont l'oubli a d'ailleurs causé deux incidents instructifs (chapitres 11 et 20.7). Une seule commande, `docker compose up -d`, démarre l'ensemble de la pile : c'est, littéralement, toute la procédure d'installation du projet.

Le cas macOS et Colima — Les conteneurs Docker reposent sur des fonctionnalités du noyau Linux (namespaces, cgroups) : sur macOS, une machine virtuelle Linux intermédiaire est indispensable pour héberger le démon Docker. La solution officielle, Docker Desktop, s'est révélée inutilisable : la version de macOS de la machine de développement (Ventura 13.7.8) est sortie de la fenêtre de compatibilité des versions récentes de Docker Desktop, dont l'installation était refusée (chapitre 20.1) — et dont la licence est par ailleurs devenue payante pour un usage en entreprise. L'environnement repose donc sur Colima (« Containers on Linux on Mac »), une alternative open source et gratuite installée via Homebrew : elle crée une VM Linux légère via Lima, en s'appuyant sur le framework de virtualisation natif de macOS, et y fait tourner le démon Docker. Le client docker habituel communique ensuite avec ce démon via le socket `~/colima/default/docker.sock`, de façon totalement transparente : toutes les commandes docker et docker compose fonctionnent à l'identique, sans interface graphique ni contrainte de version d'OS. Seul point de vigilance au quotidien : la VM ne démarre pas automatiquement avec la machine — sans `colima start` préalable, toute commande Docker échoue avec « Cannot connect to the Docker daemon ». Trois environnements coexistent ainsi : le développement local (Colima + Docker Compose, quatre conteneurs), l'intégration continue (GitHub Actions, qui exécute les tests sur une base SQLite en mémoire, sans conteneurs à démarrer), et la production (Render, un conteneur unique combinant nginx et PHP-FPM, adossé aux services PostgreSQL et Redis managés — chapitre 19.3).

Partie III — Conception Merise

Chapitre 5 — Dictionnaire de données

5.1 Recensement des entités

Quatre entités structurent le domaine : UTILISATEUR, LIVRE, EMPRUNT et SESSION_LECTURE.

5.2 Tableau du dictionnaire

Attribut	Entité	Type	Contraintes
id	users	UUID	PK, gen_random_uuid()
email	users	VARCHAR(180)	UNIQUE, NOT NULL
password_hash	users	TEXT	NOT NULL, défaut "
extra_shelves	users	INT	NOT NULL, défaut 0
title / author	books	VARCHAR	NOT NULL
status	books	reading_status (ENUM)	to_read / reading / finished / lent
cover_path	books	VARCHAR(255)	chemin de la couverture uploadée
google_volume_id	books	VARCHAR(64)	référence Gutendex si applicable
borrower	loans	VARCHAR(180)	NOT NULL (texte libre)
returned_at	loans	TIMESTAMPTZ	NULL tant que non rendu
started_at / ended_at	reading_sessions	TIMESTAMPTZ	NOT NULL / NULLABLE

Chapitre 6 — MCD (Modèle Conceptuel de Données)

6.1 Règles de gestion

- Un utilisateur possède zéro à plusieurs livres ; un livre appartient à exactement un utilisateur.
- Un livre peut faire l'objet de zéro à plusieurs emprunts.
- Un livre peut faire l'objet de zéro à plusieurs sessions de lecture.
- L'emprunteur n'est pas un utilisateur du système : c'est un attribut texte libre de l'association EMPRUNT.

6.2 Diagramme MCD

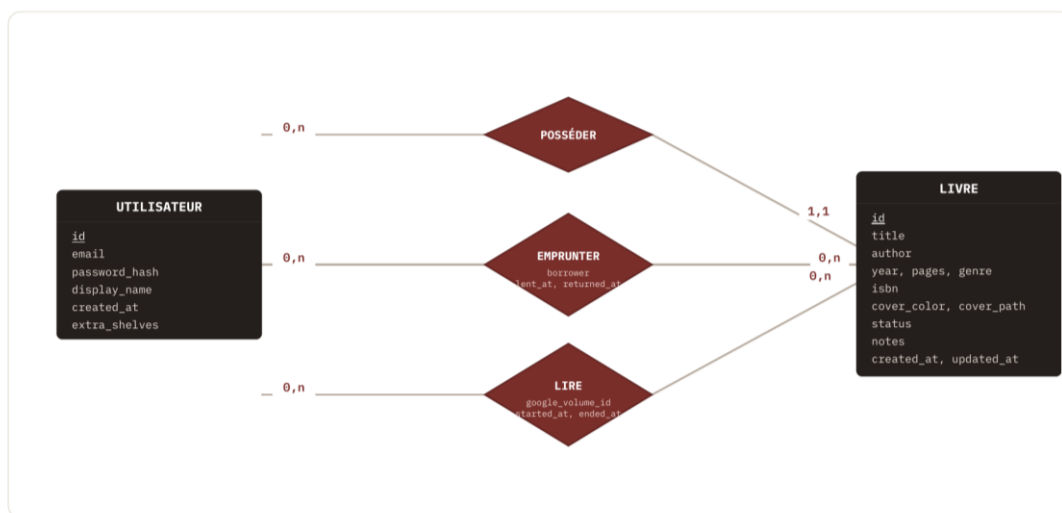


Figure 1 — MCD de BookNest

6.3 Explication des entités et associations

L'entité UTILISATEUR porte l'identification et le compteur d'étagères supplémentaires (`extra_shelves`). L'association POSSÈDE ($1,1 / 0,N$) relie UTILISATEUR et LIVRE. ENREGISTRE relie UTILISATEUR et EMPRUNT ; CONCERNE relie LIVRE et EMPRUNT. Cette dernière association est en réalité dénormalisée au niveau physique (la clé utilisateur est reportée directement dans EMPRUNT plutôt que déduite via LIVRE) pour accélérer la requête « tous mes emprunts en cours » sans jointure.

Chapitre 7 — MLD (Modèle Logique de Données)

7.1 Règles de passage MCD → MLD

Chaque entité devient une table. Chaque association (1,1)–(0,N) reporte la clé primaire du côté 1 vers le côté N sous forme de clé étrangère.

7.2 Schéma MLD

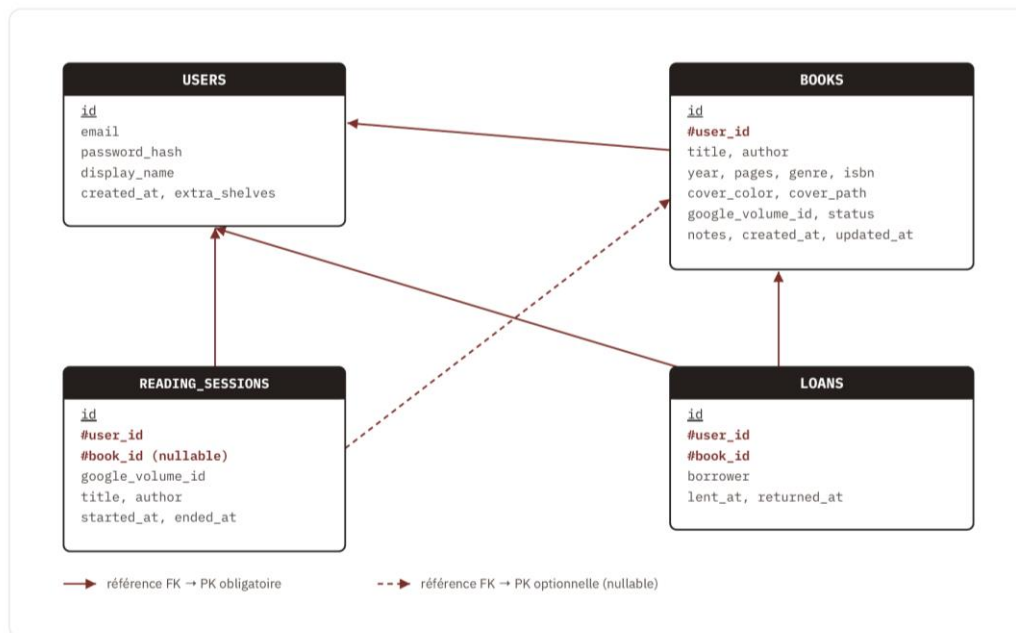


Figure 2 — MLD de BookNest

UTILISATEUR(id, email, password_hash, display_name, created_at, extra_shelves)

LIVRE(id, #user_id, title, author, year, pages, genre, isbn, cover_color, cover_path, google_volume_id, status, notes, created_at, updated_at)

EMPRUNT(id, #user_id, #book_id, borrower, lent_at, returned_at)

SESSION_LECTURE(id, #user_id, #book_id, google_volume_id, title, author, started_at, ended_at)

7.3 Description des relations

Toutes les clés étrangères appliquent ON DELETE CASCADE : la suppression d'un utilisateur supprime automatiquement ses livres, emprunts et sessions de lecture, garantissant l'intégrité référentielle sans logique applicative additionnelle.

Chapitre 8 — MPD (Modèle Physique de Données)

8.1 Types PostgreSQL retenus

- UUID pour toutes les clés primaires, générées via `gen_random_uuid()` (extension `pgcrypto`).
- ENUM `reading_status` pour le statut du livre (`to_read`, `reading`, `finished`, `lent`).
- `TIMESTAMPTZ` pour tout horodatage, avec fuseau.

8.2 Schéma MPD

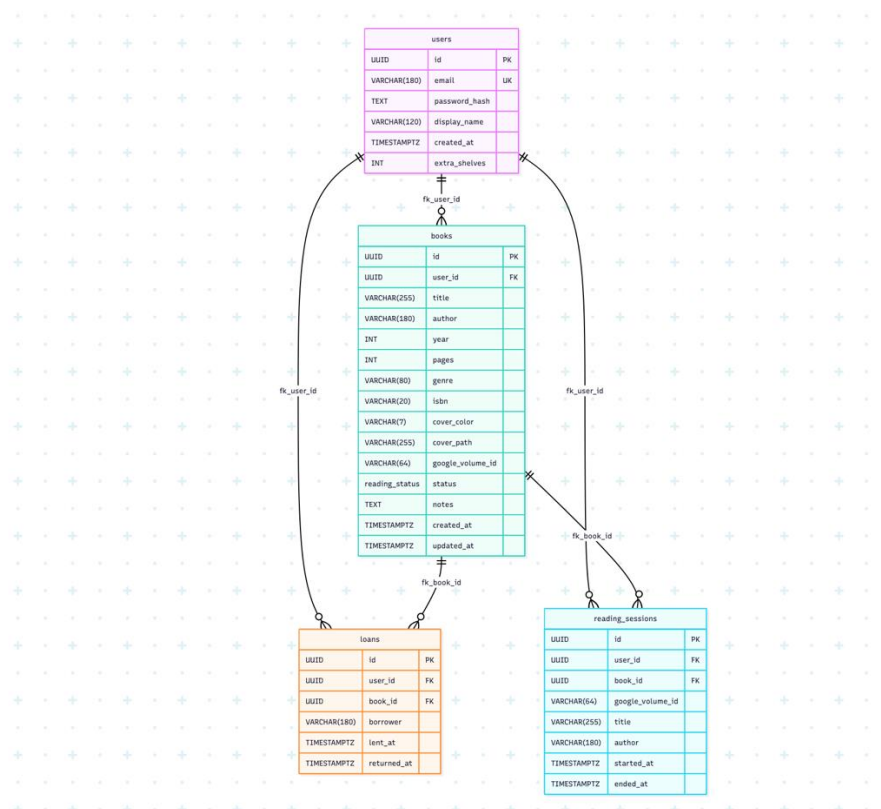


Figure 3 — MPD PostgreSQL

8.3 Index et contraintes

Le schéma évolue via des migrations SQL numérotées séquentiellement (001 à 010), rejouables intégralement sur une base vierge. Trois index accélèrent les requêtes fréquentes : `idx_books_user` (livres d'un utilisateur), `idx_books_status` (filtrage par statut), `idx_loans_user_active` (emprunts en cours d'un utilisateur). Principe retenu : ne jamais modifier une migration déjà appliquée — toute correction de schéma passe par une nouvelle migration.

Chapitre 9 — Architecture MVC

Le pattern Model-View-Controller sépare les responsabilités : le Repository encapsule l'accès aux données (aucune requête SQL n'est jamais construite directement dans un Controller) ; le Controller reçoit la requête, orchestre Repository et Service, prépare la réponse ; la View, un simple gabarit PHP, se contente d'afficher les données transmises.

MVC est un patron d'architecture, pas un outil : n'importe quel framework PHP (Laravel, Symfony...) l'implémente en fournissant déjà le routeur, la couche Request/Response et les middlewares. BookNest fait le choix inverse — aucun framework n'est utilisé. Router.php, Request.php, Response.php et AuthenticateMiddleware.php sont écrits entièrement à la main, sans dépendre d'une couche d'abstraction existante. Le patron est le même que dans un framework ; c'est la mécanique derrière qui est ici assumée directement, dans un but pédagogique : comprendre précisément comment une requête est routée et comment un middleware l'intercepte, plutôt que de s'appuyer sur des conventions déjà résolues par un tiers.

9.2 Diagramme d'architecture

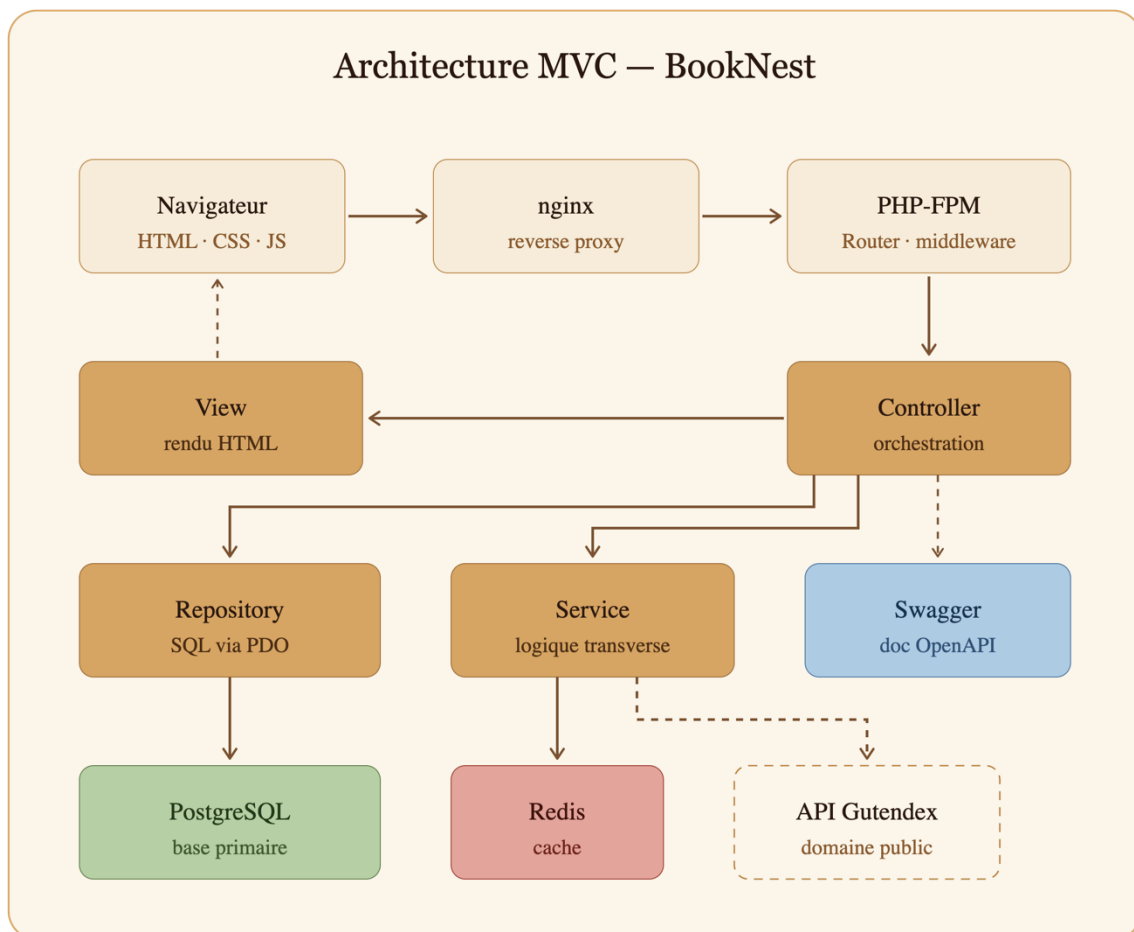


Figure 4 — Architecture MVC de BookNest

9.3 Flux d'une requête HTTP

nginx reçoit la requête et la relaie à PHP-FPM. Le Router de BookNest identifie la route par correspondance exacte de chemin, exécute le middleware d'authentification si la route est

protégée, instancie le Controller, qui délègue au Repository correspondant. Le résultat est transmis à la View pour rendu HTML — pas d'API JSON séparée pour les pages, seules quelques routes exposent du JSON pur (`/db-test`).

9.3 Le routeur

```
public function dispatch(string $uri, string $method): void
{
    $uri = parse_url($uri, PHP_URL_PATH) ?? '/';

    foreach ($this->routes[$method] ?? [] as $path => $callback) {
        if ($path === $uri) {
            $callback();
            return;
        }
    }

    http_response_code(404);
    echo '404 - Page not found';
}
```

Explication : Le matching est strict (comparaison exacte de chaîne), sans support de segments dynamiques type `/books/{id}`. Cette limite assumée a orienté certaines routes vers des paramètres en query string plutôt qu'en chemin (par exemple `/library/read?volume_id=...` plutôt que `/library/read/{id}`).

9.4 Modèles

Contrairement aux Repositories (logique d'accès aux données propre à un contrôleur), les Models exposent des méthodes statiques réutilisables autour d'une entité, notamment pour l'authentification :

```
public static function allByUser(string $userId): array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('
        SELECT id, user_id, title, author, year, pages, genre, isbn, cover_color, status, notes, created_at, updated_at
        FROM books
        WHERE user_id = :user_id
        ORDER BY created_at DESC
    ');
    $stmt->execute(['user_id' => $userId]);

    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

public static function findById(string $id, string $userId): ?array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('
        SELECT id, user_id, title, author, year, pages, genre, isbn, cover_color, status, notes, created_at, updated_at
        FROM books
        WHERE id = :id AND user_id = :user_id
        LIMIT 1
    ');
    $stmt->execute([
        'id' => $id,
        'user_id' => $userId
    ]);

    $book = $stmt->fetch(PDO::FETCH_ASSOC);
    return $book ? : null;
}
```

Explication : User::findByEmail() est utilisé par AuthController::login() pour vérifier les

```
public static function create(array $data): array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('
        INSERT INTO books (user_id, title, author, year, pages, genre, isbn, cover_color, status, notes)
        VALUES (:user_id, :title, :author, :year, :pages, :genre, :isbn, :cover_color, :status, :notes)
        RETURNING id, user_id, title, author, year, pages, genre, isbn, cover_color, status, notes, created_at, updated_at
    ');

    $stmt->execute([
        'user_id' => $data['user_id'],
        'title' => $data['title'],
        'author' => $data['author'],
        'year' => $data['year'] ?? null,
        'pages' => $data['pages'] ?? null,
        'genre' => $data['genre'] ?? null,
        'isbn' => $data['isbn'] ?? null,
        'cover_color' => $data['cover_color'] ?? '#7a2e2a',
        'status' => $data['status'] ?? 'to_read',
        'notes' => $data['notes'] ?? null,
    ]);

    return $stmt->fetch(PDO::FETCH_ASSOC);
}
```

identifiants. User::create() utilise la clause RETURNING de PostgreSQL pour récupérer l'enregistrement fraîchement inséré en un seul aller-retour base, sans SELECT supplémentaire.

```
public static function findByEmail(string $email): ?array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('SELECT * FROM users WHERE email = :email LIMIT 1');
    $stmt->execute(['email' => $email]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    return $user ?? null;
}

public static function findById(string $id): ?array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('SELECT id, email, display_name, created_at FROM users WHERE id = :id LIMIT 1');
    $stmt->execute(['id' => $id]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    return $user ?? null;
}
```

Explication : findById() prend systématiquement \$userId en second paramètre, combiné à la clause WHERE : un utilisateur ne peut jamais accéder à la fiche d'un livre appartenant à quelqu'un d'autre, même en devinant un UUID valide — la vérification d'appartenance est intégrée à la requête elle-même.

Chapitre 10 — Structure des dossiers

10.1 Arborescence du projet

```
booknest/
├── Dockerfile
├── README.md
├── app/
│   ├── Config/          → Database.php, OpenApi.php
│   ├── Controllers/     → Auth, Book, Dashboard, Home, Library, Loan
│   ├── Core/            → App.php, Request.php, Response.php, Router.php
│   ├── Middleware/      → AuthenticateMiddleware, MiddlewareInterface
│   ├── Models/          → Book.php, User.php
│   ├── Repositories/    → Author, Book, Home, Loan, ReadingSession, User
│   ├── Services/        → CacheService, GutendexService, JwtService
│   └── Views/           → auth/, books/, home/, layouts/, library/,
                        loans/, pages/, users/
├── composer.json / composer.lock
├── database/migrations/ → 001 à 010, SQL versionné
├── docker/              → nginx.conf
├── docker-compose.yml
├── index.php · router.php
├── phpunit.xml
├── public/
│   ├── assets/{css,images,js}
│   ├── openapi.json
│   ├── swagger/         → Swagger UI statique
│   └── uploads/books/    → couvertures uploadées (ignoré par Git)
└── tests/              → suite PHPUnit
```

35 dossiers, 100 fichiers (hors vendor/ et .git/)

10.2 Rôle de chaque dossier

public/ est la seule racine exposée par nginx — protège tout le reste du code d'un accès direct via l'URL. app/Core contient le socle du framework maison (routage, requête/réponse). app/Repositories isole toute logique SQL, jamais mêlée aux contrôleurs. database/migrations trace l'évolution du schéma dans le temps de façon rejouable.

Chapitre 11 — Base de données PostgreSQL

11.1 Connexion et migrations

La connexion PDO est centralisée dans une classe Database dédiée. Les identifiants (hôte, port, base, utilisateur, mot de passe) proviennent de variables d'environnement lues via un .env, chargé au bootstrap de l'application.

11.2 Transactions

Les opérations impliquant plusieurs écritures liées — par exemple créer un emprunt tout en passant le statut du livre à « lent » — sont encapsulées dans une transaction PDO explicite (beginTransaction / commit / rollBack), garantissant qu'aucun état partiel ne subsiste en cas d'erreur.

```
$pdo->beginTransaction();

try {
    $insert = $pdo->prepare("
        INSERT INTO loans (user_id, book_id, borrower, lent_at)
        VALUES (:user_id, :book_id, :borrower, now())
    ");
    $insert->execute([
        'user_id' => $_SESSION['user']['id'],
        'book_id' => $bookId,
        'borrower' => $borrower,
    ]);

    // Le statut 'lent' existe dans l'ENUM reading_status
    $update = $pdo->prepare("
        UPDATE books SET status = 'lent', updated_at = now()
        WHERE id = :id AND user_id = :user_id
    ");
    $update->execute([
        'id' => $bookId,
        'user_id' => $_SESSION['user']['id'],
    ]);

    $pdo->commit();
} catch (\Throwable $e) {
    $pdo->rollBack();
}
```

Explication : Si l'une des deux écritures échoue, rollBack() annule l'ensemble : jamais un emprunt créé sans que le livre ne passe au statut « prêté », ni l'inverse.

Chapitre 12 — Cache Redis

12.1 Stratégie de cache

Redis sert exclusivement de cache : aucune donnée métier n'y réside sans exister d'abord en PostgreSQL. La stratégie est cache-aside — on tente d'abord la lecture du cache ; en cas d'absence, on interroge la base et on écrit le résultat avec un TTL.

12.2 Un client RESP réimplémenté

Plutôt que de dépendre de l'extension PECL redis comme boîte noire, le client a été réimplémenté directement au-dessus d'un socket TCP brut, en suivant le protocole RESP (Redis Serialization Protocol) — choix pédagogique pour comprendre le protocole sous-jacent.

```
public function remember(string $key, callable $callback, int $ttl = 300): mixed
{
    // cache-aside : on tente d'abord de lire le cache,
    // puis on génère et stocke la donnée si elle n'existe pas.
    $cached = $this->get($key);
    if ($cached !== null) {
        return $cached;
    }

    $value = $callback();
    $this->set($key, $value, $ttl);

    return $value;
}
```

Explication : `remember()` est la méthode principale utilisée par les contrôleurs : elle rend transparente la présence ou non du cache pour l'appelant. Si Redis est indisponible (connexion échouée au bootstrap), toutes les méthodes publiques échouent silencieusement (retour `null/false`), sans jamais lever d'exception ni interrompre la requête — un principe de résilience validé par les tests unitaires.

Chapitre 13 — Sécurité et Authentification

13.1 JWT + session hybride

BookNest combine un token JWT (stocké en cookie, signé HS256) et une session PHP classique, reconstruite à chaque requête à partir du token. Ce choix hybride bénéficie de la portabilité d'un JWT tout en gardant la simplicité d'accès de `$_SESSION` dans le reste du code.

```
public static function createToken(array $user, int $ttl = self::DEFAULT_TTL): string
{
    $secret = self::getSecret();
    $issuedAt = time();

    $payload = [
        'iss' => 'booknest',
        'sub' => (string) ($user['id'] ?? ''),
        'email' => $user['email'] ?? '',
        'display_name' => $user['display_name'] ?? '',
        'iat' => $issuedAt,
        'nbf' => $issuedAt,
        'exp' => $issuedAt + $ttl,
    ];

    return JWT::encode($payload, $secret, 'HS256');
}
```

Explication : Le TTL par défaut est de 86400 secondes (24h). Passé ce délai, tout token présenté est automatiquement rejeté par la bibliothèque firebase/php-jwt lors du décodage.

13.2 Middleware de protection des routes

```
public function handle(Request $request): void
{
    if (!in_array($request->uri, $this->protectedPaths, true)) {
        return;
    }

    if (!empty($_SESSION['user'])) {
        return;
    }

    header('Location: /login', true, 303);
    exit;
}
```

Explication : Le code HTTP 303 (plutôt que 301/302) force le navigateur à rejouer la requête suivante en GET, quelle que soit la méthode d'origine — pertinent car une route protégée comme `/books/delete` est en POST, et on ne veut pas qu'une redirection préserve accidentellement cette méthode vers `/login`.

13.3 Sécurité des données

- Mots de passe hachés via `password_hash()` (bcrypt par défaut), jamais stockés en clair.
- Requêtes préparées PDO systématiques — aucune concaténation de variable dans une requête SQL.
- Secrets (mots de passe base de données) externalisés via variables d'environnement, jamais codés en dur dans `docker-compose.yml`.

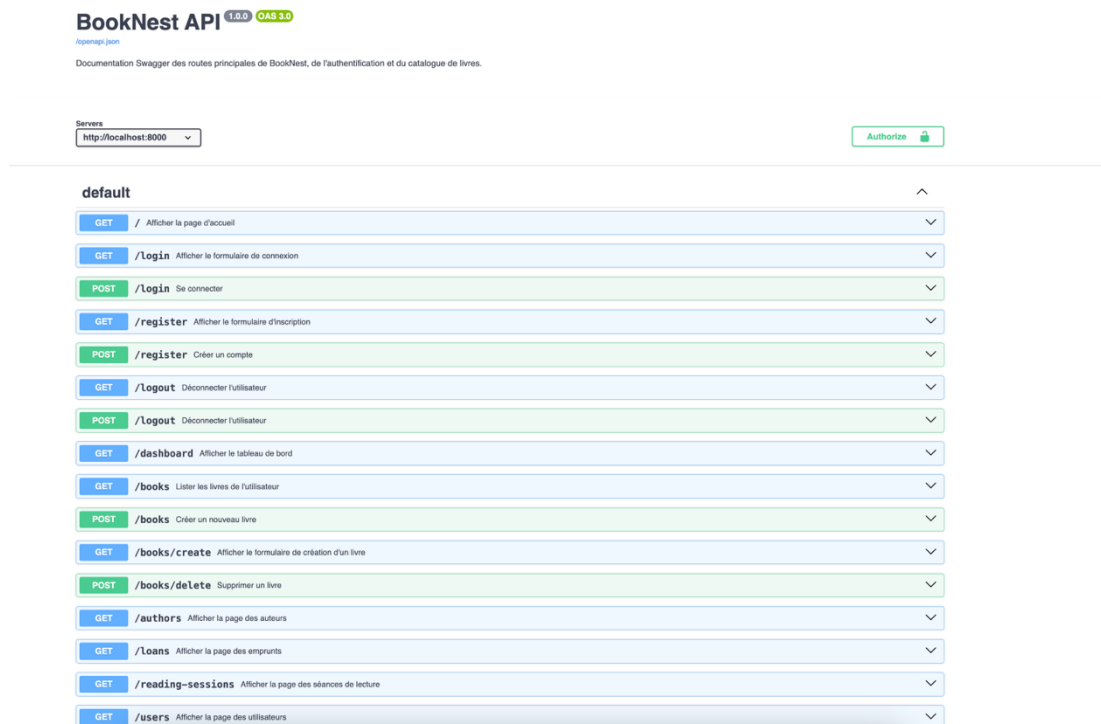
Chapitre 14 — API et Documentation Swagger

14.1 Génération depuis les annotations

Chaque route est documentée via des annotations PHPDoc directement dans les contrôleurs, scannées par zircote/swagger-php pour générer public/openapi.json, exposé via une interface Swagger UI statique à /swagger/.

```
/**
 * @OA\Post(
 *   path="/books/update-status",
 *   tags={"Books"},
 *   summary="Changer le statut de lecture d'un livre",
 *   @OA\RequestBody(
 *     required=true,
 *     @OA\JsonContent(
 *       required={"book_id","status"},
 *       @OA\Property(property="book_id", type="string", example="123e4567-e89b-12d3-a456-426614174000"),
 *       @OA\Property(property="status", type="string", enum={"to_read","reading","finished"})
 *     )
 *   ),
 *   @OA\Response(response=303, description="Statut mis à jour, redirection vers l'accueil")
 * )
 */
```

Explication : L'annotation vit au plus près du code qu'elle documente : toute modification de la route est immédiatement visible dans la documentation régénérée, sans fichier de spec séparé à maintenir manuellement.



Interface Swagger UI

14.2 Endpoints principaux

- POST /api/auth/register — inscription.
- POST /api/auth/login — obtention du JWT.
- GET /api/books — liste avec recherche et filtres.
- POST /api/books — ajout d'un livre (auth).
- GET /api/books/{id} — fiche détaillée.
- POST /api/loans — enregistrement d'un prêt.
- GET /api/stats — statistiques de lecture (cache 60 s).

Partie V — Front-end BookNest

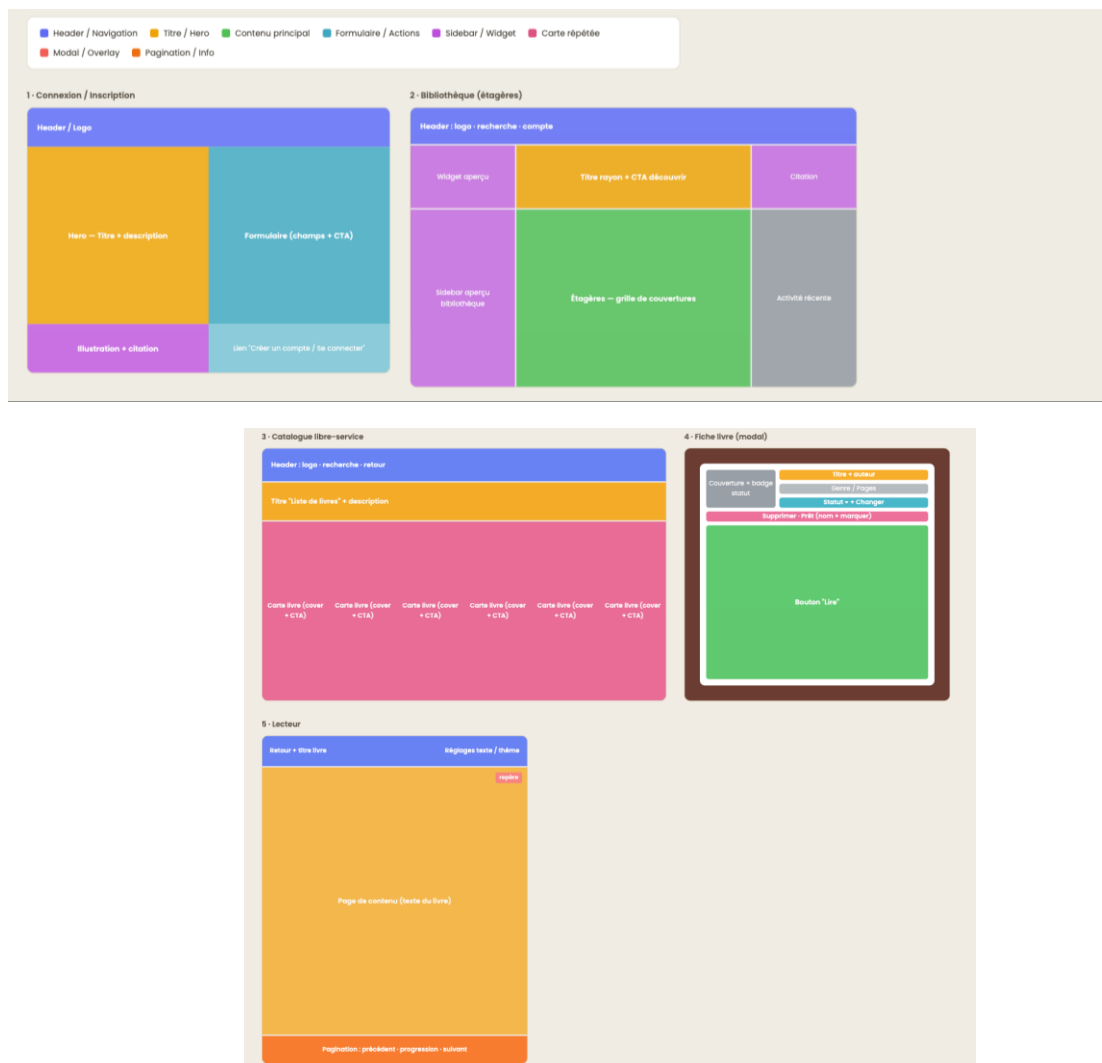
Chapitre 15 — Direction artistique

15.1 Une identité assumée

Plutôt qu'une interface générique de gestion de données, BookNest adopte une identité visuelle de bibliothèque ancienne : tons bois et laiton, étagères avec relief 3D (grain du bois, leur chaude ambiante), typographie à deux voix — Cormorant Garamond (serif) pour les titres et le texte du lecteur, Inter (sans-serif) pour l'interface.

15.2 Zoning

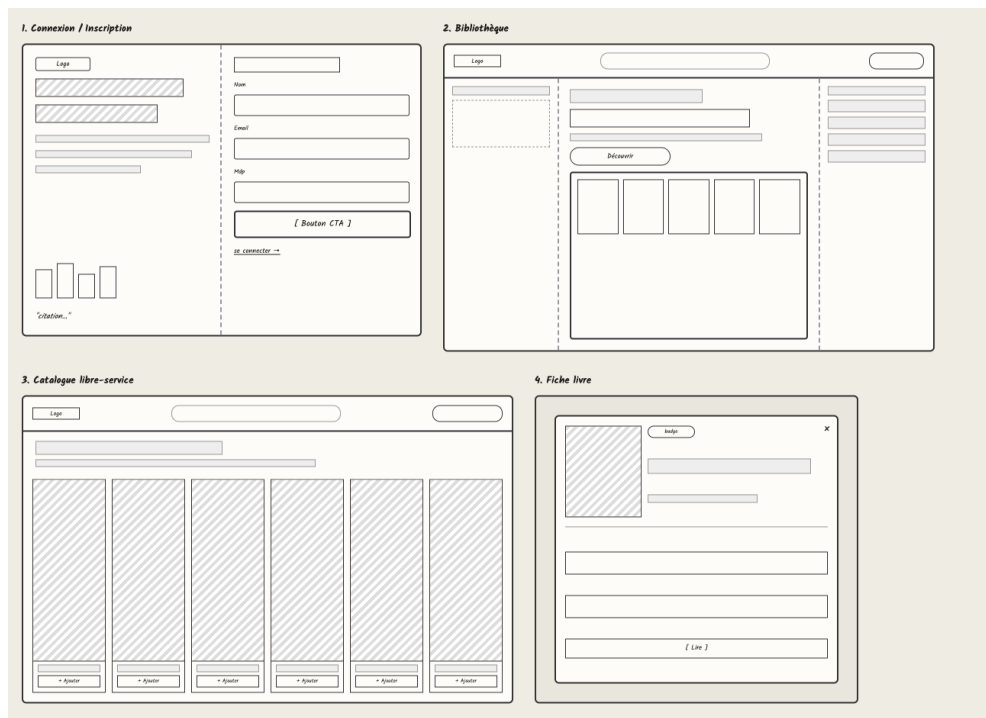
Découpage fonctionnel des écrans clés avant maquettage : chaque couleur représente un type de zone (header/navigation, titre/hero, contenu principal, formulaire/actions, sidebar/widget, carte répétée, modal/overlay, pagination/info), cohérent d'un écran à l'autre.



connexion/inscription et bibliothèque, catalogue livre service, fiche livre, lecteur

15.3 Wireframes

Structure basse-fidélité des 5 écrans principaux, sans couleur ni contenu final — pour valider l'agencement avant l'habillage visuel :

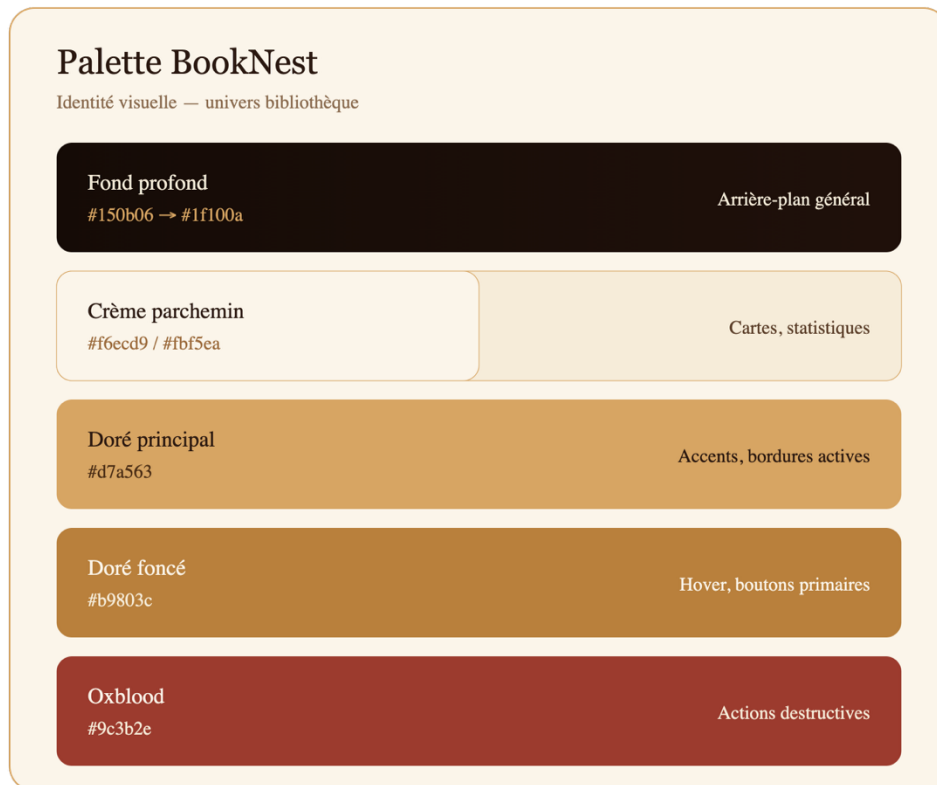


Wireframes : connexion, bibliothèque, catalogue, fiche livre

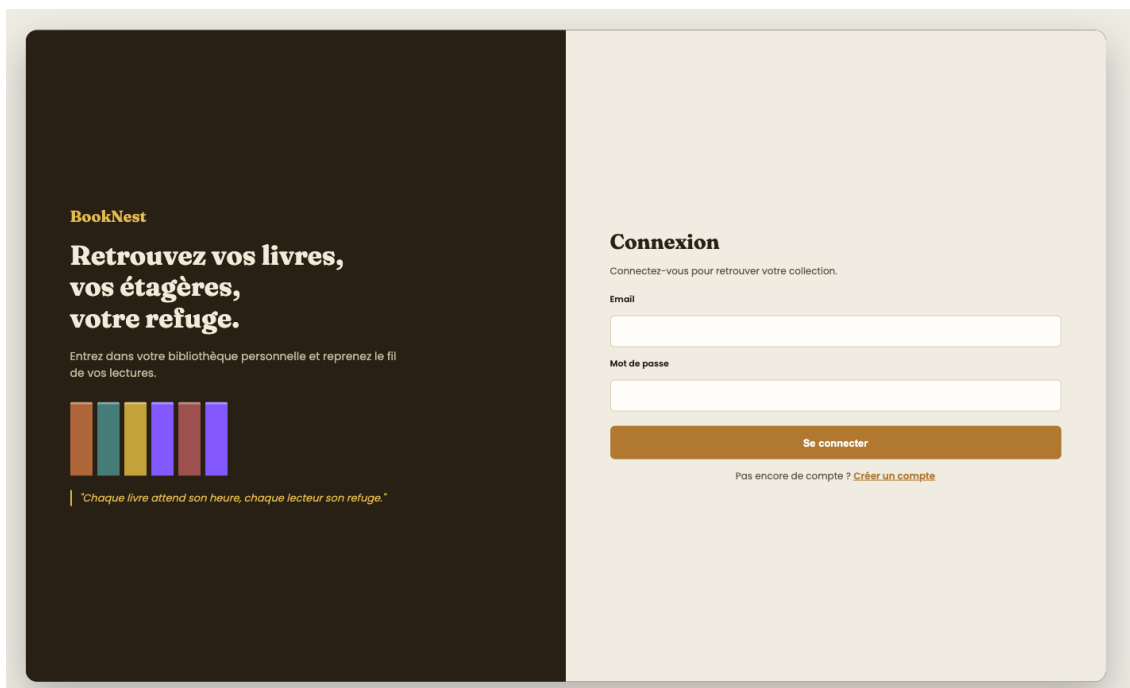


Wireframe du lecteur

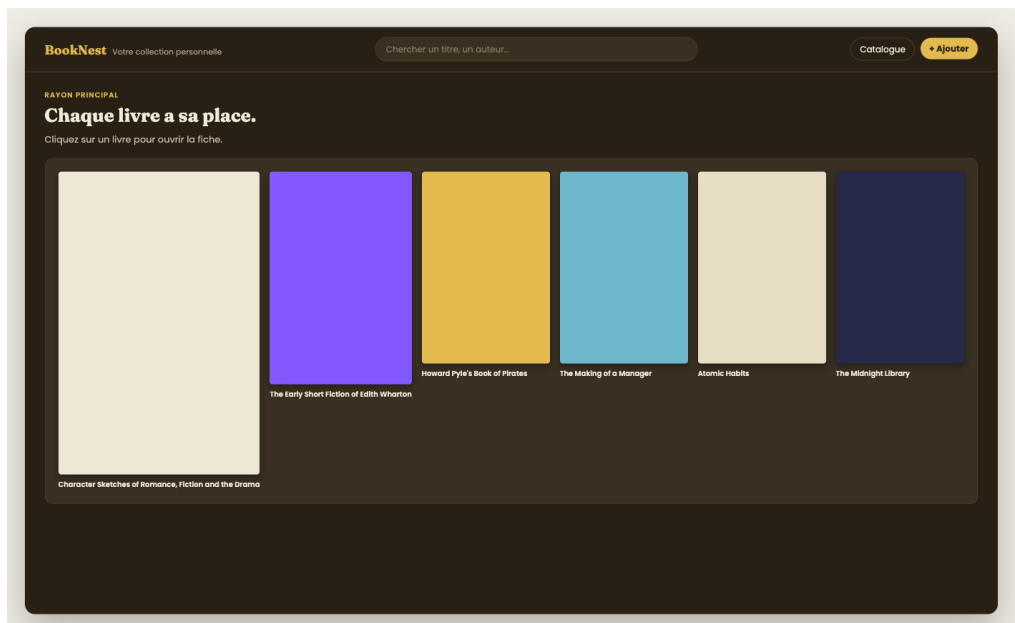
15.4 Palette de couleurs



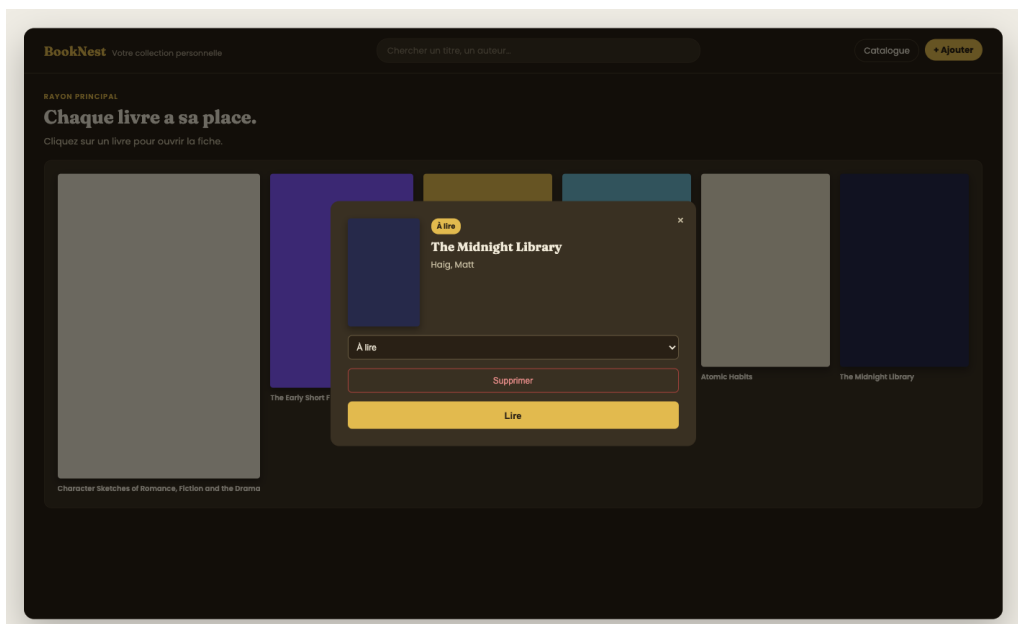
15.5 Prototype



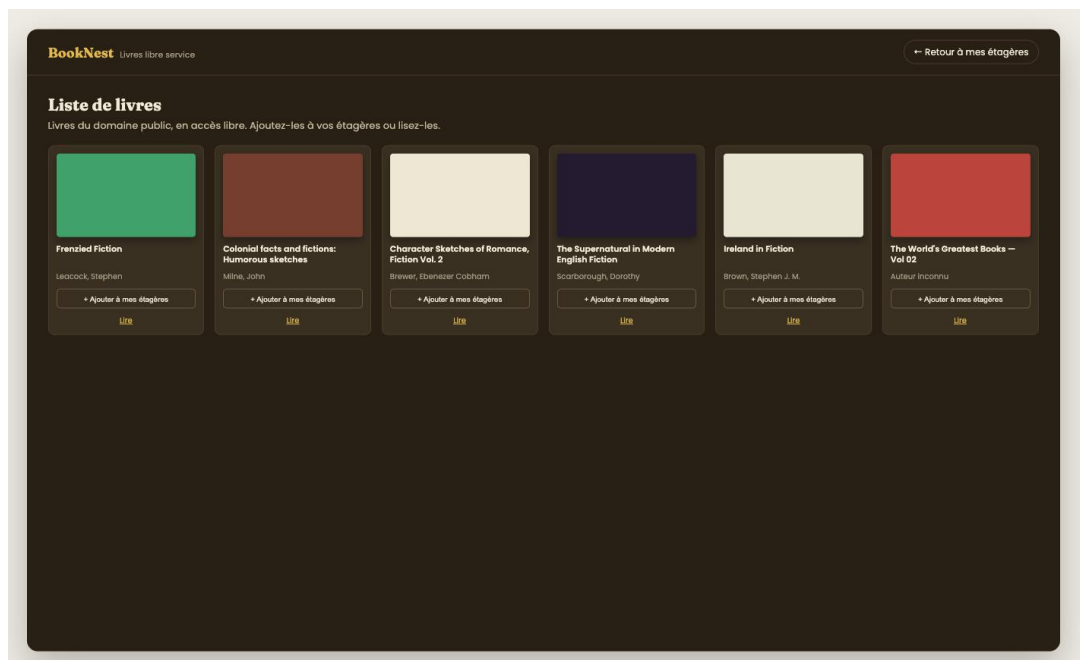
Page de connexion



Page d'accueil (étagères)



Fiche livre (modal)



Catalogue libre service

Chapitre 16 — Écrans principaux

16.1 Page d'accueil — étagères

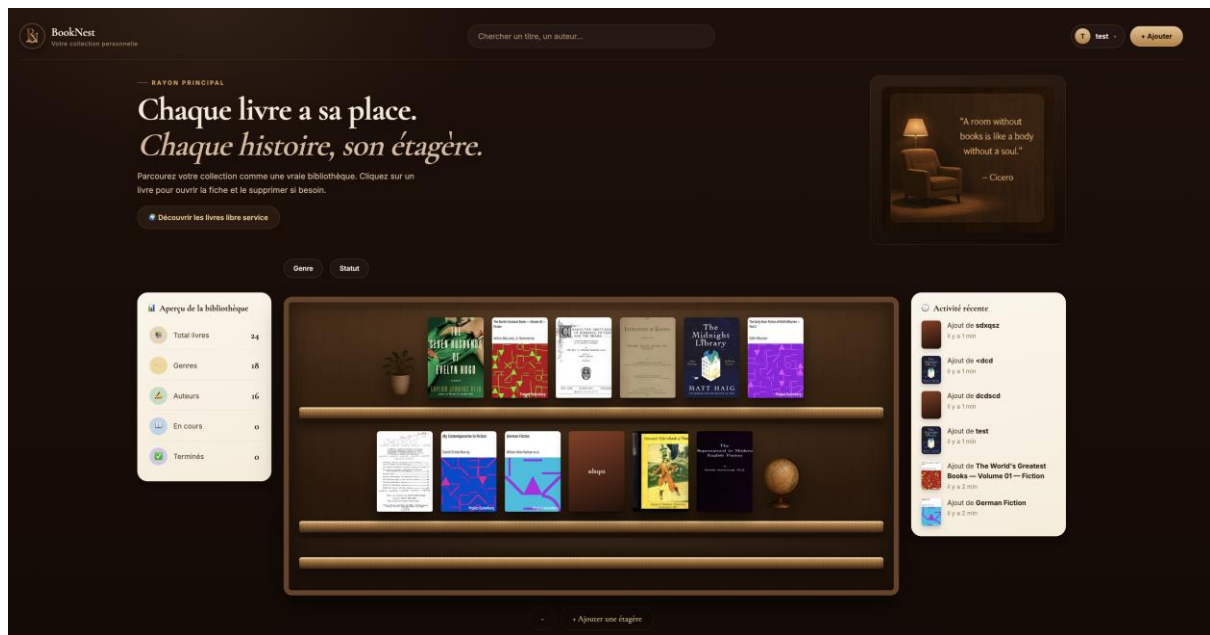


Figure 5 — Page d'accueil : bibliothèque en rayons

La page d'accueil dispose les livres sur plusieurs rayons.

16.2 Fiche livre

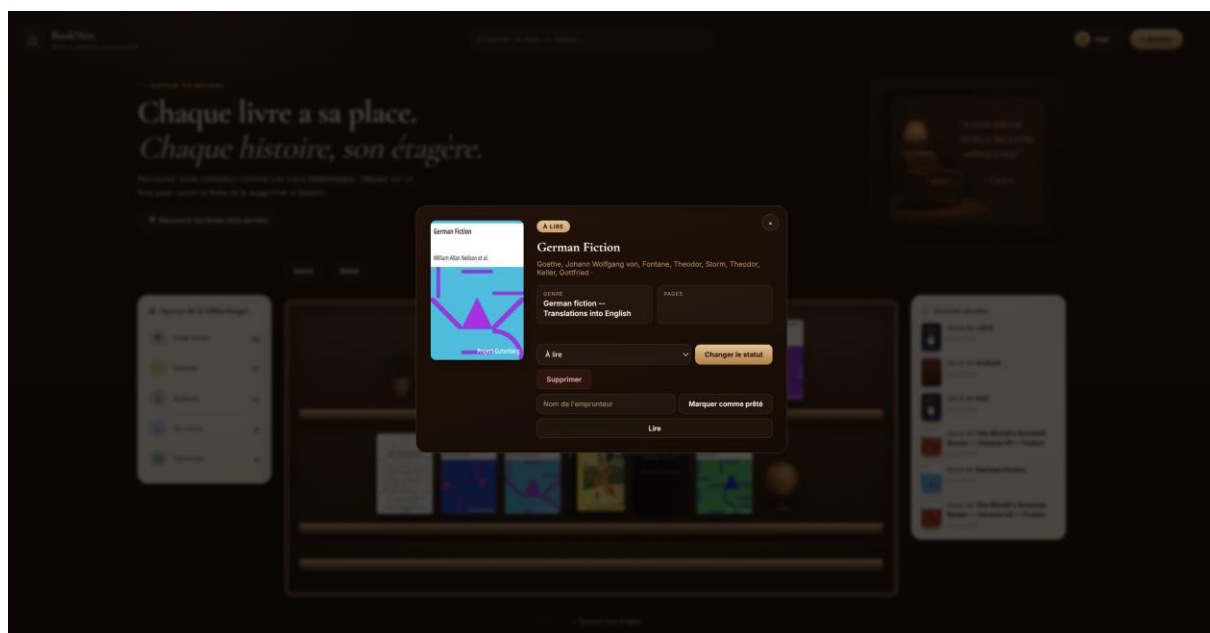


Figure 6 — Fiche livre au clic sur une tranche

Le dialogue affiche une couverture miniature, l'auteur, l'année, le statut, les notes personnelles, et propose des actions : lire le livre, changer de statut, supprimer, emprunt.

16.3 Lecteur immersif

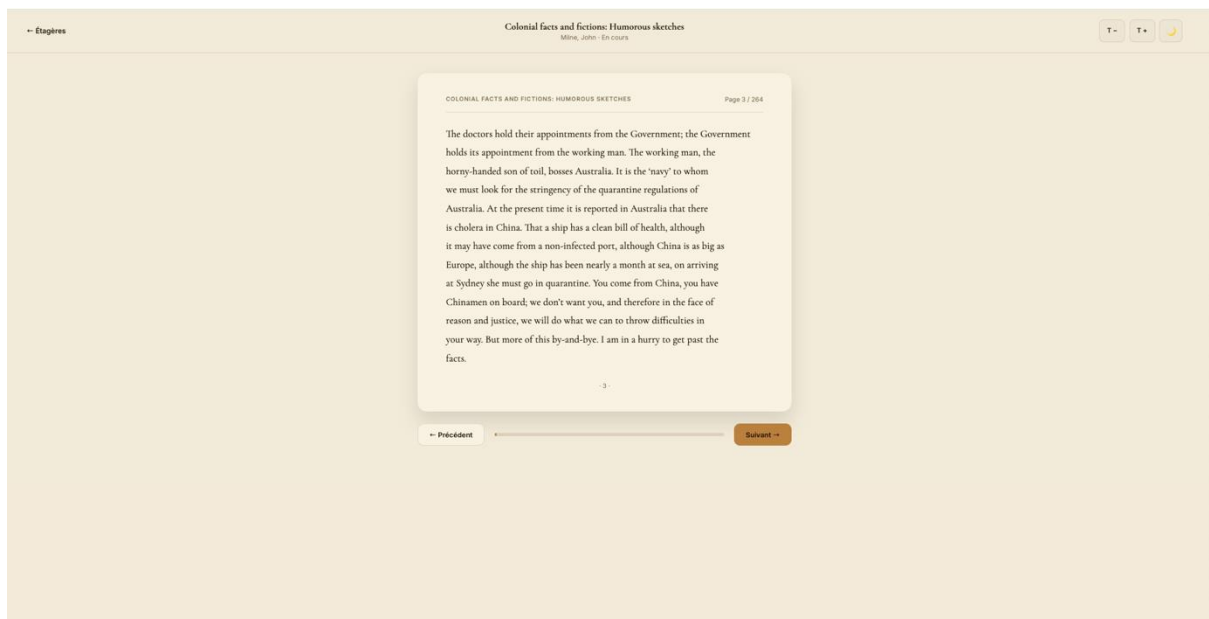


Figure 7 — Lecteur paginé avec thèmes

Le lecteur découpe le texte en pages, propose deux thèmes (sépia et nuit), un curseur de taille de police, et un marque-page qui reprend la couleur de la tranche du livre lu.

16.4 Formulaire d'ajout

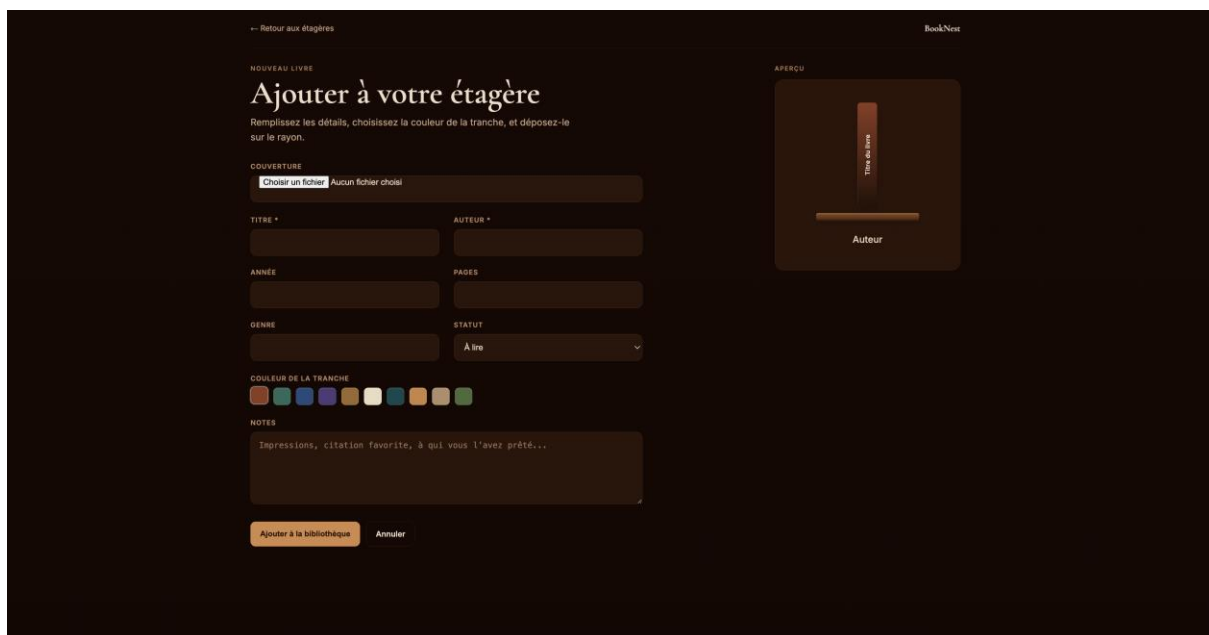
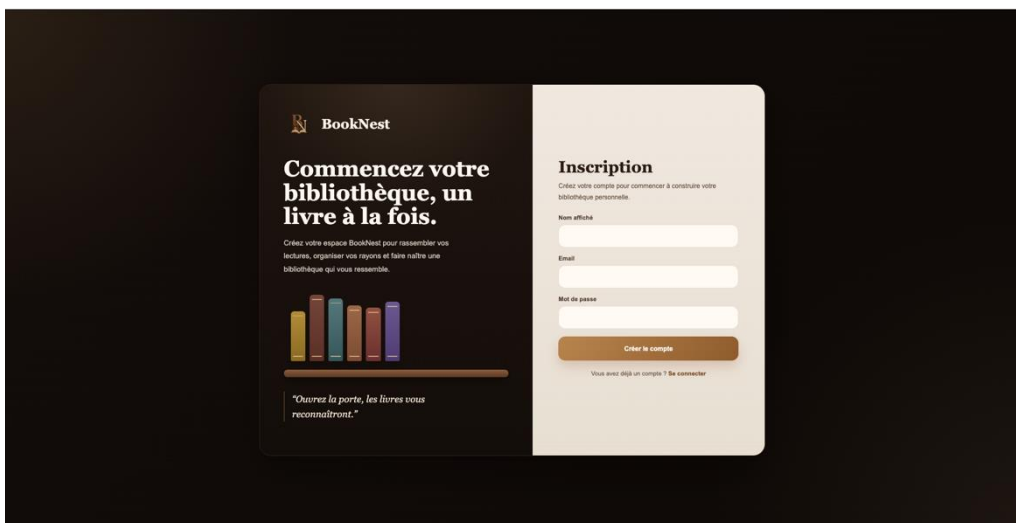
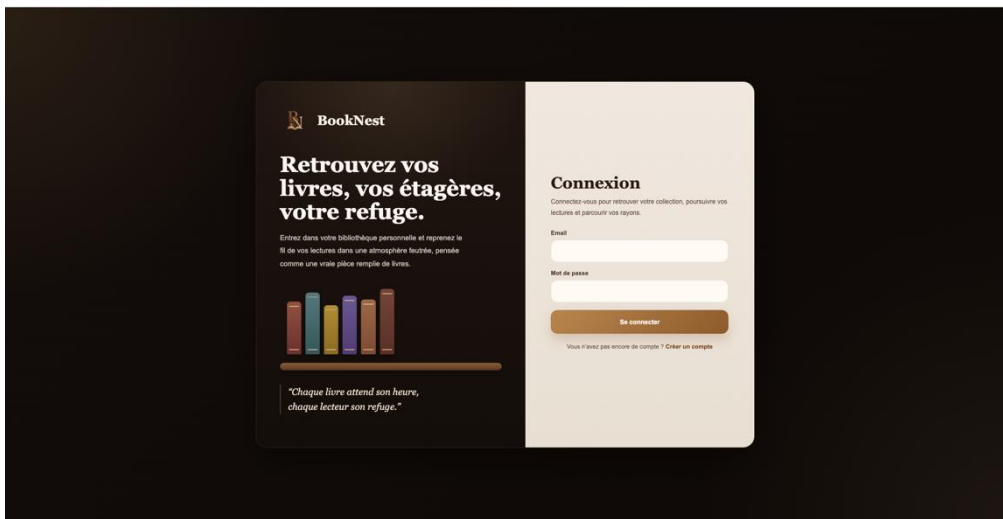


Figure 8 — Formulaire avec aperçu

16.5 Page de connexion/Inscription



Partie VI — Qualité & Livraison

Chapitre 17 — Tests

17.1 Tests automatisés

Suite PHPUnit couvrant les composants isolables du projet, avec une base SQLite en mémoire pour les repositories (pas de dépendance à PostgreSQL pour les tests) :

Fichier	Composant	Ce qui est vérifié
BookRepositoryTest.php	BookRepository	Création + lecture, suppression d'un livre et de sa couverture
CacheServiceTest.php	CacheService	Dégradation gracieuse si Redis indisponible
JwtServiceTest.php	JwtService	Cycle création/décodage, rejet d'un token invalide
RouterTest.php	Router	Dispatch correct, retour 404 pour route inconnue

Résultats des tests unitaires

```
Time: 00:00.215, Memory: 8.00 MB

Book Repository
✓ Create and all
✓ Delete removes row and file

Cache Service
✓ Cache unavailable returns null
✓ Remember uses callback when cache unavailable

Jwt Service
✓ Create and decode token
✓ Decode invalid token returns null

Router
✓ Dispatch calls correct route
✓ Dispatch returns 404 for unknown route

OK (8 tests, 19 assertions)
```

17.2 Cahier de recette

Authentification

Test	Objectif	Données d'entrée	Résultat attendu
Inscription	Créer un compte	Email, mot de passe, nom	Compte créé, redirection accueil
Connexion invalide	Rejeter	Mauvais mot de passe	Message d'erreur, pas de session
Déconnexion	Fermer la session	—	Session détruite, redirection /login

Livres & Emprunts

Test	Objectif	Données d'entrée	Résultat attendu
Ajout de livre	Créer un livre	Titre, auteur obligatoires	Livre visible sur l'étagère
Changement de statut	Modifier le statut	Nouveau statut valide	Livre déplacé sur la bonne étagère
Marquer comme prêté	Créer un emprunt	Nom de l'emprunteur	Emprunt créé, statut livre → lent
Marquer comme rendu	Clôturer un emprunt	ID du livre	returned_at renseigné, statut → to_read

Catalogue libre-service & étagères

Test	Objectif	Données d'entrée	Résultat attendu
Recherche Gutendex	Interroger l'API	Terme de recherche	Liste de résultats affichée
Ajout à l'étagère	Importer un livre externe	volume_id Gutendex	Livre créé avec google_volume_id
Lecture paginée	Charger le texte	volume_id + page	Page affichée, session de lecture démarrée
Ajouter une étagère	Incrémenter extra_shelves	—	Une étagère vide supplémentaire apparaît

Chapitre 18 — Gestion Git

18.1 Organisation des branches

Le projet est versionné sur GitHub. La branche par défaut est main ; le développement se fait sur develop et sur des branches de fonctionnalité dédiées, fusionnées progressivement :

main(branche par défaut, remote)

feature/library(branche active à date de rédaction)

```
core/openapi-router
db/migrations
develop
feat/home-profile-logout
feat/home-search
feat/library-architecture
feature/bootstrap
feature/bootstrap-mvc
feature/library
middleware/authentication
```

Explication : Chaque branche isole une préoccupation précise : mise en place du squelette MVC (feature/bootstrap*), authentification (middleware/authentication), évolution du schéma (db/migrations), routage et documentation Swagger (core/openapi-router), puis les fonctionnalités métier (feat/home-*, feature/library). Cette granularité permet de revenir sur une fonctionnalité isolément sans affecter le reste.

18.2 Convention de commits

feat(library): switch to Gutendex full-text reader instead of Google Books widget
feat(loans): add loan creation and return workflow
feat(home): redesign layout with overview/activity columns and shelf toolbar
fix(router): instantiate controller before route registration
chore(docker): increase nginx client_max_body_size for cover uploads

Explication : Cette granularité (feat/fix/chore) permet de retracer précisément l'évolution de chaque fonctionnalité et facilite un retour en arrière ciblé si nécessaire.

Chapitre 19 — Déploiement

19.1 Docker Compose

Explication : Les secrets (mot de passe PostgreSQL) sont externalisés via des variables d'environnement lues depuis `.env` (non versionné), plutôt que codés en dur — une correction apportée en cours de projet après revue du fichier.

```
services:
  >Run Service
  app:
    build: .
    volumes:
      - ./var/www/html:delegated
    depends_on:
      - db
    environment:
      - APP_ENV=development
    networks:
      - booknet

  >Run Service
  web:
    image: nginx:stable-alpine
    ports:
      - "8000:80"
    volumes:
      - ./var/www/html:ro
      - ./docker/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app
    networks:
      - booknet

  >Run Service
  db:
    image: postgres:13
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - db-data:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    networks:
      - booknet

  >Run Service
  redis:
    image: redis:7
    ports:
      - "6379:6379"
    networks:
      - booknet

volumes:
  db-data:

networks:
  booknet:
    driver: bridge
```

19.2 Migrations versionnées

Le schéma évolue via des scripts SQL numérotés (001 à 010), rejouables intégralement sur une base vierge :

```
for f in database/migrations/*.sql; do
  docker compose exec -T db psql -U postgres -d booknest < "$f";
done
```

19.3 Hébergement en ligne et intégration continue

Au-delà de l'environnement de développement local (Docker Compose sur la machine de développement), l'application est déployée sur Render, une plateforme d'hébergement gratuite pour les projets personnels. Le déploiement repose sur trois éléments :

- Un Dockerfile dédié (Dockerfile.render), qui combine nginx et PHP-FPM dans un seul conteneur via supervisord — Render exécute un conteneur unique par service, contrairement à Docker Compose en local qui orchestre plusieurs conteneurs séparés
- Un fichier render.yaml (Infrastructure as Code) qui décrit les trois services nécessaires — l'application web, une base PostgreSQL et un cache Redis — permettant de recréer tout l'environnement Render en un clic plutôt que de configurer chaque service manuellement
- Un pipeline d'intégration continue GitHub Actions qui exécute la suite de tests PHPUnit à chaque envoi sur la branche principale, et ne déclenche le déploiement sur Render que si l'ensemble des tests passe

```
name: CI/CD — Tests puis déploiement Render
on:
  push:
    branches: [main]

jobs:
  test:
    steps:
      - run: composer install
      - run: vendor/bin/phpunit --testdox

  deploy:
    needs: test
    if: success()
    steps:
      - run: curl -f "${{ secrets.RENDER_DEPLOY_HOOK_URL }}"
```

Explication : Le déploiement n'est jamais déclenché directement : il dépend systématiquement de la réussite préalable des tests automatisés (needs: test, if: success()). Une régression détectée par la suite PHPUnit bloque donc la mise en production avant même d'atteindre Render — une démarche DevOps où la qualité conditionne la livraison plutôt que d'être vérifiée après coup.

Limite assumée du plan gratuit Render : la base PostgreSQL expire automatiquement 30 jours après sa création (contrainte du tier gratuit), nécessitant de rejouer les migrations sur une base recréée périodiquement pour un usage prolongé au-delà de la démonstration.

Partie VII — Bilan

Chapitre 20 — Difficultés rencontrées et solutions

Cette section retrace, dans l'ordre chronologique, une chaîne de difficultés réellement rencontrées — depuis la remise en cause du choix d'API pour le catalogue libre service jusqu'aux bugs de performance et de cache découverts lors des derniers tests utilisateur. Elle illustre une démarche de résolution de problème complète : symptôme observé, diagnostic, correction, vérification.

20.1 Incompatibilité Docker Desktop / macOS

Problème : installation de Docker Desktop refusée (« Impossible d'utiliser cette version de l'application Docker avec cette version de macOS »), Docker Desktop ne supportant qu'un nombre limité de versions macOS récentes.

Solution : bascule vers Colima, moteur Docker léger basé sur une VM Lima, sans contrainte de version d'OS — installé simplement via Homebrew.

20.2 Bascule d'API externe pour le catalogue libre service

L'intégration initiale reposait sur l'API Google Books, avec lecture via son widget officiel. Problèmes rencontrés : quota anonyme rapidement épuisé, catalogue de résultats peu pertinent (vieux fonds de bibliothèque scannés plutôt que de véritables œuvres), et certains résultats passant le filtre « gratuit » sans être réellement lisibles intégralement (viewability différent de ALL_PAGES, widget affichant « Acheter ce livre »).

Solution : bascule complète vers l'API Gutendex (Project Gutenberg) — gratuite, sans clé ni quota, donnant accès au texte intégral brut de véritables œuvres du domaine public. Développement d'un lecteur de texte paginé fait maison plutôt que de dépendre d'un widget tiers : plus de contrôle sur le découpage en pages, plus de fiabilité.

20.3 Cache navigateur masquant les mises à jour CSS

Problème : après modification de la feuille de style, l'ancienne version restait affichée malgré un rafraîchissement forcé, à cause d'un en-tête Cache-Control trop agressif sur les fichiers statiques.

```
<link rel="stylesheet" href="/assets/css/home.css?v=<?= filemtime(
__DIR__ . '/../..'/public/assets/css/home.css'
)?>">
```

Explication : Un paramètre de version basé sur la date de modification du fichier force le navigateur à recharger dès que le CSS change, sans intervention manuelle future.

20.4 Perte de la base de données après renommage du projet

Problème : renommage du dossier du projet (booknest → ProjetCDA), puis erreur PDOException relation "users" does not exist au premier chargement.

Diagnostic : le nom du dossier fait partie du nom des volumes Docker générés automatiquement par Docker Compose (booknest_db-data devient projetcda_db-data). Le renommage a donc silencieusement créé un tout nouveau volume vide, sans lien avec les données existantes — les migrations n'y avaient jamais été rejouées.

Solution : rejouer l'intégralité des migrations sur le nouveau volume :

```
for f in database/migrations/*.sql; do
  docker compose exec -T db psql -U postgres -d booknest < "$f";
done
```

20.5 Doublons de livres importés depuis le catalogue

Problème : un même livre du catalogue libre service se retrouvait parfois dupliqué dans la bibliothèque (deux lignes distinctes pour le même google_volume_id).

Diagnostic : LibraryController::add() insérait systématiquement une nouvelle ligne, sans vérifier si l'utilisateur avait déjà ce livre — un double-clic sur « Ajouter » suffisait à créer un doublon.

Solution : vérification de l'existence (user_id + google_volume_id) avant toute insertion, avec redirection silencieuse si le livre est déjà présent.

20.6 Lenteur perçue de la recherche et de la lecture

Problème : ouvrir le catalogue libre service ou un livre à lire prenait plusieurs secondes, à chaque fois, y compris pour un livre déjà consulté.

Diagnostic : LibraryController::index() et ::read() appelaient l'API Gutendex en direct à chaque requête, sans aucune mise en cache — y compris pour changer simplement de page dans un livre déjà chargé.

Solution : mise en cache des résultats de recherche (10 min) et des métadonnées de livre (24h) via CacheService, complétée par un stockage des métadonnées en session dès le premier accès — les changements de page (Précédent/Suivant) ne déclenchent alors plus aucun appel réseau ni requête de cache.

20.7 Cache Redis silencieusement inactif

Problème : après la correction précédente, la lenteur persistait exactement comme avant. Un test direct du cache (CacheService::set() puis ::get()) retournait systématiquement null.

Diagnostic : la variable d'environnement REDIS_HOST n'était pas définie dans .env. CacheService retombait alors sur sa valeur par défaut 127.0.0.1 — qui, depuis l'intérieur du conteneur app, pointe vers le conteneur lui-même et non vers le conteneur redis. Un problème de configuration identique, dans son principe, à celui déjà rencontré avec DB_HOST en tout début de projet (Chapitre 11).

Solution : ajout de REDIS_HOST=redis et REDIS_PORT=6379 dans .env — sur le réseau Docker interne, le nom du service tient lieu d'adresse.

20.8 Corruption du cache sur les grosses valeurs (texte de livre)

Problème : une fois Redis correctement configuré, la recherche du catalogue devenait rapide, mais la lecture d'un livre restait défaillante.

Diagnostic : le client Redis du projet réimplémente le protocole RESP directement au-dessus d'un socket TCP (Chapitre 12). Or `fwrite()` et `fread()` sur un socket PHP ne garantissent pas de traiter la totalité d'un payload en un seul appel — une limite sans conséquence pour de petites valeurs (résultats de recherche), mais critique pour le texte intégral d'un livre, potentiellement plusieurs centaines de Ko.

```
// avant : un seul appel, silencieusement tronqué sur les gros payloads
$payload = fread($this->socket, $length + 2);

// après : boucle jusqu'à avoir lu exactement $length + 2 octets
$toRead = $length + 2;
$payload = "";
while (strlen($payload) < $toRead) {
    $chunk = fread($this->socket, $toRead - strlen($payload));
    if ($chunk === false || $chunk === "") { $this->available = false; return false; }
    $payload .= $chunk;
}
```

Explication : Le même principe est appliqué en écriture (`writeAll()`). Cette catégorie de bug — une limite d'API système qui ne se manifeste qu'au-delà d'une certaine taille de donnée — est particulièrement instructive : un test avec de petites valeurs ne l'aurait jamais révélée.

Chapitre 21 — Perspectives d'évolution

- Étendre la recherche sur les livres via l'extension `pg_trgm` (déjà installée, non encore exploitée par un index).
- Ajouter des tests d'intégration et end-to-end, au-delà des tests unitaires actuels.
- Mettre en place un pipeline CI/CD (GitHub Actions) exécutant la suite PHPUnit à chaque push.
- Étoffer le suivi des sessions de lecture (durée, pages lues) au-delà du simple horodatage début/fin.
- Reconstituer une charte graphique et des maquettes formalisées a posteriori, pour documenter la démarche UX/UI attendue.

Conclusion générale

BookNest démontre qu'une application web professionnelle peut être à la fois techniquement rigoureuse — MVC strict, PostgreSQL, Redis, JWT, Swagger, tests PHPUnit — et visuellement chaleureuse. Le pari initial était de fuir l'esthétique CRUD au profit d'une métaphore immersive : le résultat, avec ses étagères boisées et son lecteur paginé, valide pleinement cette intuition.

Sur le plan pédagogique, ce projet a permis de mettre en pratique l'ensemble du cycle Merise, la conception d'une API REST documentée, la gestion Git professionnelle et le déploiement conteneurisé. Les perspectives d'évolution (import ISBN, mobile, recommandations) ouvrent de nombreuses pistes pour prolonger le travail.

BookNest est prêt à accueillir vos livres — et, qui sait, à devenir votre bibliothèque de tous les jours.

Bibliographie

- PHP: The Right Way — <https://phptherightway.com>
- Documentation PostgreSQL 13 — <https://www.postgresql.org/docs/13/>
- Documentation Redis / protocole RESP — <https://redis.io/docs/latest/develop/reference/protocol-spec/>
- Gutendex API — <https://gutendex.com>
- Spécification OpenAPI 3.0 — <https://spec.openapis.org/oas/v3.0.3>
- Merise — Nanci, Espinasse, Cohen, éd. Vuibert.

